Forever Hotel

Software Design Specification (SDS)

By

Rashmika Nethsarani
SE/2021/017
Rasadhi Sanchala
SE/2021/021
Hiruni Ramanayaka
SE/2021/024
Viduni Niketha
SE/2021/044
Pubudu Madushan
SE/2021/060

A report submitted in partial fulfilment of the requirements for the degree of Bachelor of Science Honors in Software Engineering (B. Sc.SE)

Software Engineering Teaching Unit
Faculty of Science
University of Kelaniya
Sri Lanka
2026

Version History Table

Version No.	Date	Author(s)	Summary of Changes
1.0	2026.04.18	Rashmika Nethsarani Rasadhi Sanchala Hiruni Ramanayaka Viduni Niketha Pubudu Madushan	Initial SDS draft

TABLE OF CONTENTS

CHAPTER 1: ARCHITECTURAL DESIGN 1
1.1 High-Level Architecture 1
1.2 Cloud-Native and Containerized Architecture 3
1.2.1 Container Deployment Model 3
1.2.2 Infrastructure Specifications 4
1.3 Justification for Architecture Choice 4
1.4 Component Communication Design 5
1.4.1 Synchronous REST Communication 6
1.4.2 Asynchronous Event-Driven Communication 6
1.4.3 Real-Time WebSocket Communication 7
1.5 Scalability and Maintainability Considerations 8
1.5.1 Scalability 8
1.5.2 Maintainability 9
CHAPTER 2: DATABASE DESIGN 10
2.1 ER Diagram 10
2.2 Normalization explanation 11
2.2.1 Normal Forms Applied 11
2.2.2 From Unnormalized Form (UNF) to 1NF 12
2.2.3 Second Normal Form (2NF) 15
2.2.4 Third Normal Form (3NF) Transitive Dependency Analysis 16
2.2.5 Boyce-Codd Normal Form (BCNF) 17
2.2.6 Anomaly Elimination 18
2.2.7 Normalized Schema Summary 19
CHAPTER 3: CLASS DIAGRAM 22

CHAPTER 4: SEQUENCE DIAGRAMS 23
4.1 Online Room Booking 23
4.2 Guest Check-in 24
4.3 Guest Check-out 25
4.4 Worker Registration 26
4.5 Worker Complaint Management 27
4.6 Manager Analytics 28
4.7 Manage Promotions 29
4.8 Order Food 30
4.9 Request Service 31
4.10 Worker Task Claim 32
CHAPTER 5: UI/UX DESIGN 33
5.1 Wireframes or screen mock-ups for key user interfaces 33
5.2 User Flow and Navigation Diagrams 33
5.2.1 Hotel Website (HW) - User Flows 33
5.2.2 Manager Analytical Dashboard (MAD) - User Flows 38
5.2.3 Front Desk System (FDS) - User Flows 43
5.2.4 Guest App / FOSS - User Flows 48
5.2.5 Kitchen Management System (KMS) - User Flows 52
5.2.6 Worker Management System (WKMS) - User Flows 55
CHAPTER 6: SECURITY DESIGN 60
6.1 Authentication Mechanism 60
6.1.1 JWT-Based Authentication 60
6.1.2 Guest Authentication Flow 61
6.1.3 Staff Authentication Flow 61

6.1.4 Password Storage 62
6.2 Authorization Levels 63
6.2.1 Role Definitions 63
6.2.2 Permission Validation Process 64
6.3 Data Protection Approach 64
6.3.1 Payment Data Protection (PCI-DSS Compliance) 64
6.3.2 Personally Identifiable Information (PII) Protection 66
6.3.3 Audit Logging 66
6.4 Input Validation and Sanitization Strategy 67
6.4.1 Validation Layers 68
6.4.2 Validation Rules by Data Type 68
6.4.3 SQL Injection Prevention 69
6.4.4 Cross-Site Scripting (XSS) Prevention 69
6.5 Encryption Standards for Data at Rest and in Transit 69
6.5.1 Encryption in Transit 69
6.5.2 TLS Certificate Management 70
6.5.3 Encryption at Rest 70
6.5.4 Database Column Encryption 71
CHAPTER 7: DEPLOYMENT DESIGN 72
7.1 Hosting Plan 72
7.2 Hardware Requirements 73
7.2.1 Server Hardware (VPS) 73
7.2.2 On-premises Hardware 74
7.3 Network Diagram 75
7.4 Environment Specification 76

7.5 CI/CD Pipeline Overview 77
7.5.1 Pipeline Stages 78
7.5.2 Pipeline Implementation Details 78
7.5.3 Independent Service Deployment 80
CHAPTER 8: REFERENCES 81

LIST OF FIGURES

Figure 1- Forever Hotel System Architecture Diagram....2
Figure 2 - ER Diagram....10
Figure 3 - Class Diagram....22
Figure 4 - Online Room Booking....23
Figure 5 - Guest-Check-In....24
Figure 6 - Guest-Check-Out....25
Figure 7 - Worker Registration....26
Figure 8 - Worker Complaint Management....27
Figure 9 - Manager Analytics....28
Figure 10 - Manage Promotions....29
Figure 11 - Order Food....30
Figure 12 - Request Service....31
Figure 13 - Worker Task Claim....32
Figure 14 - HW-01: Guest Account Registration Flow....34
Figure 15 - HW-02: Guest Login Flow....35
Figure 16 - HW-03: Online Room Booking Flow....36
Figure 17 - HW-04: Booking Modification and Cancellation Flow....37
Figure 18 - MAD-05: Analytics and Operations Overview Flow....38
Figure 19 - MAD-06: Staff Account Creation and Credential Delivery Flow....39
Figure 20 - MAD-07: Staff Password Reset Flow....40
Figure 21 - MAD-08: Promotion Creation and Publish Flow....41
Figure 22- MAD-09: Complaint Resolution Flow....42
Figure 23 - MAD-10: Report Export Flow....43
Figure 24 - FDS-11: Guest Check-In (Pre-Booked and Walk-In) Flow....44
Figure 25 - FDS-12: Guest Check-Out Flow....45
Figure 26 - FDS-13: Room Change and Reassignment Flow....46
Figure 27 - FDS-14: Manual Escalation Task Assignment Flow....47
Figure 28 - FDS-15: Service Request Creation on Behalf of Guest Flow....48
Figure 29 - FOSS-16: Food Ordering Flow....50
Figure 30 - FOSS-17: Service Request Submission and Tracking Flow....50

Figure 31 - FOSS-18: Complaint Filing and Tracking Flow 51
Figure 32 - KMS-19: Order Processing Flow 53
Figure 33 - KMS-20: Menu and Stock Management Flow (Kitchen Manager) 54
Figure 34 - KMS-21: Order Cancellation Received from Guest Flow 55
Figure 35 - WKMS-22: Task Claim and Service Completion Flow 56
Figure 36 - WKMS-23: Food Delivery Task Flow 57
Figure 37 - WKMS-24: Shift Dashboard and Performance View Flow 58
Figure 38 - Guest Authentication Flow 61
Figure 39 - Staff Authentication Flow 62
Figure 40 - Payment Data Protection 65
Figure 41 - Multi-Layer Input Validation Architecture 68
Figure 42-TLS Certificate Auto-Renewal Process 70
Figure 43 - Forever Hotel Network Diagram (Production Deployment) 75
Figure 44 - CI/CD Pipeline Overview (GitHub Actions) 78

LIST OF TABLES

Table 1- Forever Hotel Subsystem Inventory 1
Table 2- Architecture Decision - Monolithic vs Microservices 4
Table 3 - Inter-Service Communication Patterns 6
Table 4 - Normal Forms 11
Table 5 - Normal Forms Applied 12
Table 6 - Third Normal Form (3NF) Transitive Dependency Analysis 16
Table 7 - Boyce-Codd Normal Form (BCNF) 18
Table 8 - Anomaly Elimination 19
Table 9 - Normalized Schema Summary 19
Table 10 - JWT Token Specifications 60
Table 11 - System Roles and Permissions 63
Table 12 - PII Protection Measures 66
Table 13 - Audit Log Events 66
Table 14 - Input Validation Specifications 68
Table 15 - Transport Encryption Specifications 70
Table 16 - Data at Rest Encryption Specifications 71
Table 17 - Hosting Plan Summary 72
Table 18 - Production Server Minimum Specifications 73
Table 19 - On Premise Hardware Requirements 74
Table 20 - Environment Specifications 76
Table 21 - CI/CD Pipeline Specifications 78

CHAPTER 1: ARCHITECTURAL DESIGN

1.1 High-Level Architecture

The Forever Hotel system is designed as a microservices-based architecture [1] comprising six independently deployable subsystems. Each subsystem addresses a distinct operational domain of Forever City Hotel and is implemented as a self-contained service with its own deployment unit and clearly defined API contract. All six subsystems share a single PostgreSQL database instance [2] to simplify deployment, reduce operational overhead, and enable efficient cross-service reporting. Tables are prefixed by subsystem identifiers to maintain logical separation while avoiding the complexity of managing multiple database clusters. Cross-cutting concerns -including authentication, request routing, and rate limiting - are centralized within a dedicated API Gateway component.

All client-facing requests, regardless of their origin (web browser, mobile device, or tablet), are routed exclusively through the API Gateway. JSON Web Token (JWT) [3] bearer tokens are validated by the gateway on all protected routes, per-IP rate limiting is enforced, and requests are routed to the appropriate downstream microservice by path prefix. Internal service-to-service communication is performed by bypassing the gateway and using direct REST calls within the [4] network.

Table 1- Forever Hotel Subsystem Inventory

Subsystem	**Technology**	**Deployment Unit**
Hotel Website (HW)	Next.js SPA	Docker container - public-facing
Manager Dashboard (MAD)	Next.js SPA	Docker container - internal network
Front Desk System (FDS)	Next.js SPA	Docker container - internal network

Subsystem	Technology	Deployment Unit
Guest App (FOSS)	Next.js PWA	Docker co ntainer - public-facing (PWA)
Kitchen Mgmt. System (KMS)	Next.js SPA	Docker container - internal network
Worker Mgmt. System (WKMS)	Next.js PWA	Docker container - internal network (PWA)

The system architecture is illustrated in Figure 1. The layered structure from client devices through the API Gateway, into the six microservices layer, and down to the shared platform services (RabbitMQ event bus, WebSocket server, and PostgreSQL database cluster) and the four external third-party integrations is depicted in the diagram.

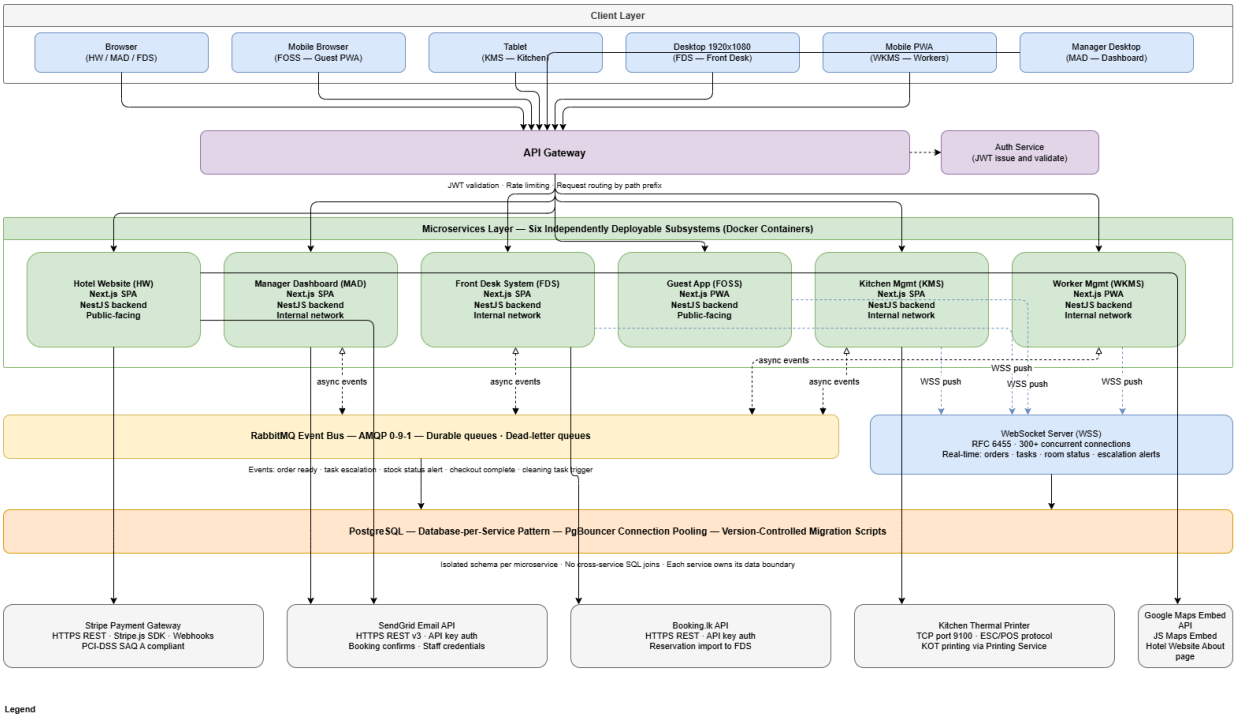


Figure 1- Forever Hotel System Architecture Diagram

1.2 Cloud-Native and Containerized Architecture

The Forever Hotel system is designed around cloud-native principles, with all microservices packaged as Docker containers and orchestrated via Docker Compose on a Linux x86-64 Virtual Private Server (VPS). Environment-agnostic, reproducible deployment on any compliant Linux host without modification to service code is ensured by this approach. Each microservice runs in an isolated container with its own process space, file system layer, and network interface. Communication between services is performed over a Docker-managed bridge network using well-defined internal hostnames, and unauthorized inter-service access outside the defined API contracts is prevented. Public-facing ports (443 for HTTPS and the WSS upgrade endpoint) are exposed only by the API Gateway container.

1.2.1 Container Deployment Model

The deployment model is structured as follows:

- API Gateway [5] - exposes HTTPS on port 443; terminates TLS [6] using Let's Encrypt certificates (auto-renewed); routes traffic to internal services by path prefix.
- Auth Service [7] - issues and validates JWT tokens; accessible only internally within the Docker bridge network.
- Six subsystem backend services (NestJS) - each runs as an independent NestJS process within its own container; exposes an internal REST API on a private port.
- Six subsystem frontend applications (Next.js) - each served as a Next.js application; guest-facing and public-facing subsystems (HW, FOSS) are accessible from the public internet via the API Gateway; staff-facing subsystems (MAD, FDS, KMS, WKMS) are accessible only from within the hotel's internal Wi-Fi network.
- RabbitMQ [8] container - provides the AMQP 0-9-1 message broker with durable queues; accessible only within the Docker bridge network.
- PostgreSQL [9] container - hosts all microservice database schemas with PgBouncer [10] connection pooling; exposed only internally.
- WebSocket server container - manages persistent WSS connections for real-time updates; accessed by clients through the API Gateway upgrade endpoint.
- Printing Service container - accepts KOT payloads from the KMS backend and dispatches ESC/POS commands to the kitchen thermal printer over raw TCP (port 9100).

1.2.2 Infrastructure Specifications

The minimum hardware requirements for the production VPS hosting the Forever Hotel system are:

- Processor: 4 CPU cores (x86-64 architecture)
- Memory: 8 GB RAM
- Storage: 50 GB SSD (for container images, database volumes, and log retention)
- Operating System: Ubuntu 22.04 LTS or later
- Network: continuous internet connectivity for public-facing subsystems (HW, FOSS); stable LAN connectivity to the hotel's thermal printer
- SSL/TLS: certificates provisioned via Let's Encrypt and auto-renewed using Certbot

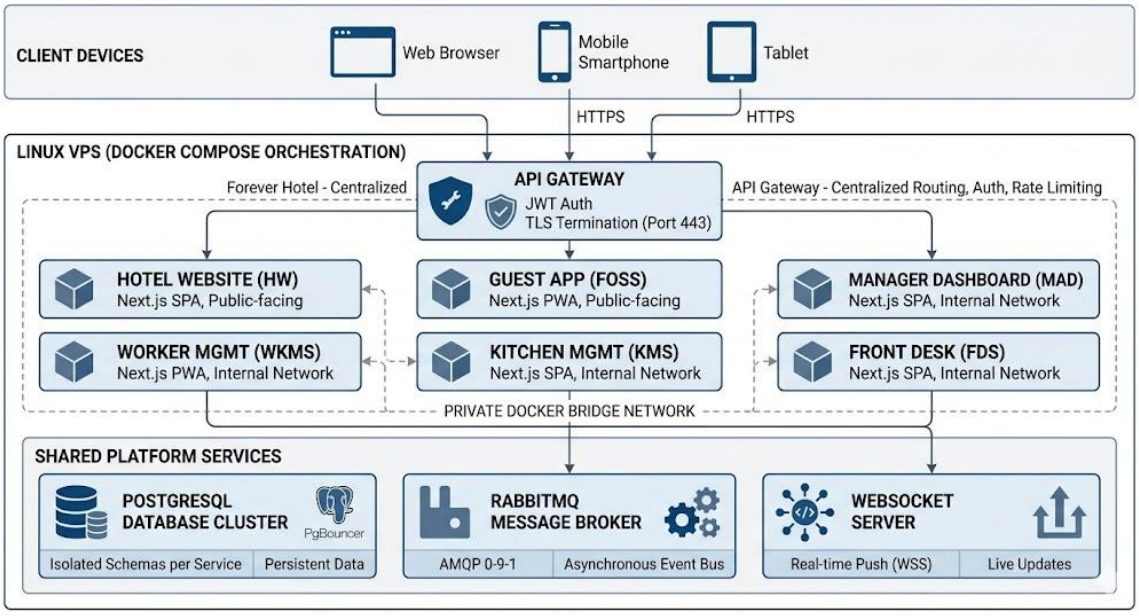


Figure 2: Infrastructure Specifications

1.3 Justification for Architecture Choice

Table 2- Architecture Decision - Monolithic vs Microservices

Dimension	**Monolithic**	**Microservices (Adopted)**
Fault isolation	A failure in one module brings down the entire system.	Container isolation ensures that a kitchen service failure does not affect the Front Desk System (NFR-06).

Dimension	Monolithic	Microservices (Adopted)
Independent deployment	Full redeployment is required for every change.	Each subsystem may be updated, restarted, or rolled back independently, enabling hotfixes with zero system-wide downtime.
Scalability	The entire application must be scaled to address any individual bottleneck.	High-load services (e.g., FOSS during peak meal periods) may be horizontally scaled independently without affecting low-load services.
Development clarity	Shared codebase introduces coupling risk as the team grows.	Clear subsystem boundaries align with team structure and reduce merge conflicts across six distinct functional domains.
Maintainability	Schema changes propagate across the entire monolith.	Shared database with table-prefix namespaces simplifies cross-service queries and reduces deployment complexity; migration scripts are versioned per service and coordinated through a central changelog to avoid conflicts

1.4 Component Communication Design

Three complementary patterns are employed for inter-service communication within the Forever Hotel system, and each pattern is selected for the specific characteristics of the interaction that it supports. The communication patterns and their application within the system are summarized in Table 3.

Table 3 - Inter-Service Communication Patterns

Pattern	**Protocol**	**Use Cases**	**Services Involved**
Synchronous (REST)	HTTPS / REST over API Gateway	User-facing requests, booking, check-in, menu browsing	All six subsystems via API Gateway
Asynchronous (events)	AMQP [11] 0-9-1 via RabbitMQ	Order ready → WKMS dispatch; task escalation → FDS; stock alert → MAD; checkout → WKMS cleaning task	KMS, WKMS, FDS, MAD, Analytics Service
Real-time push (WSS)	WebSocket Secure (RFC 6455)	Live order queue, task queue, room status board, escalation alerts	KMS, WKMS, FDS, MAD, FOSS

1.4.1 Synchronous REST Communication

Synchronous REST over HTTPS is used for all client-initiated requests that require an immediate response. Requests from browser or mobile clients are routed through the API Gateway, where the JWT bearer token is validated, rate limiting is enforced, and the request is forwarded to the appropriate downstream microservice. All REST endpoints accept and return JSON payloads encoded in UTF-8, as specified in the communications interface requirements of the SRS. Role-Based Access Control (RBAC) policies derived from the role claim embedded within the JWT are enforced on all authenticated endpoints, satisfying NFR-12.

1.4.2 Asynchronous Event-Driven Communication

Asynchronous inter-service communication is implemented via RabbitMQ using the AMQP 0-9-1 protocol [12]. The event-driven model is employed for operations where an immediate

response is not required from the consuming service by the producing service, and where strong decoupling between services is desirable. The following events are propagated via the RabbitMQ event bus:

- Order ready event - published by KMS when a food order is marked Ready; consumed by WKMS to dispatch a delivery worker (FD requirement KM05b).
- Task escalation event - published by the WKMS cron job when a task remains unclaimed for more than 15 minutes; consumed by FDS to display a notification badge (WM07b).
- Stock status change event - published by KMS when a menu item stock status is updated; consumed by MAD to display a low-stock alert (KMM06).
- Checkout completion event - published by FDS upon successful guest checkout; consumed by WKMS to create a high-priority room-cleaning task (FD10).

All RabbitMQ queues and exchanges are declared durable, ensuring that messages have persisted to disk and survive broker restarts. Dead-letter queues are implemented by each consumer service to handle processing failures without message loss, satisfying NFR-05 (100% of submitted food orders shall be persisted and recoverable after system failure).

1.4.3 Real-Time WebSocket Communication

Persistent WebSocket Secure (WSS) connections are used for all real-time push updates to client interfaces. The WebSocket server is implemented using the NestJS WebSocket gateway [13] (Socket.IO) and supports a minimum of 300 simultaneous client connections, as required by NFR-14. Exponential backoff reconnection logic with a maximum of five retry attempts is implemented by client applications, as specified in CI-02 of the SRS communications interface requirements.

Real-time updates delivered via WSS include the KMS order Kanban board (new orders, status transitions), the WKMS task queue (new tasks, claim events, escalation flags), the FDS room status board and escalation panel, the MAD operational analytics panels, and the FOSS order status tracker and service request status tracker. Updates propagate within one seconds of the triggering event, satisfying NFR-03.

- KMS order Kanban board — pushes new order arrivals and status transitions (New → In Preparation → Ready → Delivered) within 1 seconds (NFR-03)

- WKMS task queue — pushes new task arrivals, claim events by other workers, and escalating flag changes in real time
- FDS room status board and escalation panel — pushes room state changes (Vacant, Occupied, Requires Cleaning, Under Maintenance) and new escalation alerts with notification badge
- MAD analytics panels — pushes live operational data updates across booking, order, service request, and room status views
- FOSS order status and service request trackers — pushes order state transitions (Submitted → In Kitchen → Ready → Delivered) and service request state transitions (Submitted → Assigned → In Progress → Completed) to the guest's device

1.5 Scalability and Maintainability Considerations

1.5.1 Scalability

Multiple mechanisms for scaling system capacity in response to increased operational demand are provided by the microservices architecture of Forever Hotel:

- Horizontal service scaling - Individual microservice containers may be replicated behind the API Gateway load balancer during peak load periods (e.g., multiple FOSS and KMS container replicas during lunch and dinner service). The cost and complexity of scaling the entire application stack are avoided by this targeted approach.
- Database connection pooling - PgBouncer is deployed in front of the PostgreSQL server to manage connection pool exhaustion under high concurrency Connection limits are configurable per service without database schema changes.
- Event bus throughput - The addition of new consumer services without modifying existing producers is allowed by RabbitMQ's topic exchange model, and the accommodation of future operational modules without architectural revision is enabled by the system.
- WebSocket capacity - The WebSocket server is designed to support a minimum of 300 simultaneous persistent connections (NFR-14). Capacity may be increased by horizontally scaling the WebSocket container and partitioning socket namespaces by subsystem.

1.5.2 Maintainability

Direct support for long-term maintainability of the system is provided by the architectural decisions made in the design of Forever Hotel:

- Shared database with logical namespace separation - A single PostgreSQL database is shared by all six microservices. Only its own set of tables, identified by a subsystem prefix, is written to by each service. Cross-service joins are permitted for reporting and analytics but are encapsulated within read-only queries to prevent write coupling. Coordination across services is required for schema changes to avoid migration conflicts; applied scripts per service are tracked by a central migration versioning table. Deployment is simplified (one database cluster instead of six) by this approach, and operational overhead is reduced while logical isolation is still provided through naming conventions and access controls. Schema evolution is managed through version-controlled, sequentially numbered migration scripts, satisfying NFR-16.
- Defined API contracts - all inter-service REST APIs are documented with explicit request/response schemas prior to implementation. Version coordination is required for changes to API contracts, and silent breaking changes are prevented.
- Structured logging - Structured logs in JSON format are emitted by all services and are retained for a minimum of 12 months (NFR-17). Timestamp, service name, and detailed error descriptions are included in logs (NFR-18), and rapid incident diagnosis across the distributed system is enabled.
- Environment-parity - Identical service images are ensured by Docker containers for the development, staging, and production environments, so environment-specific bugs are eliminated and the time required to reproduce and resolve defects is reduced.
- Independent release cadence - Because each subsystem is independently deployable, bug fixes in the Kitchen Management System, for example, may be released and verified in production without coordinating with or testing the other five subsystems. The release overhead for a five-person development team is substantially reduced by this approach.

CHAPTER 2: DATABASE DESIGN

2.1 ER Diagram

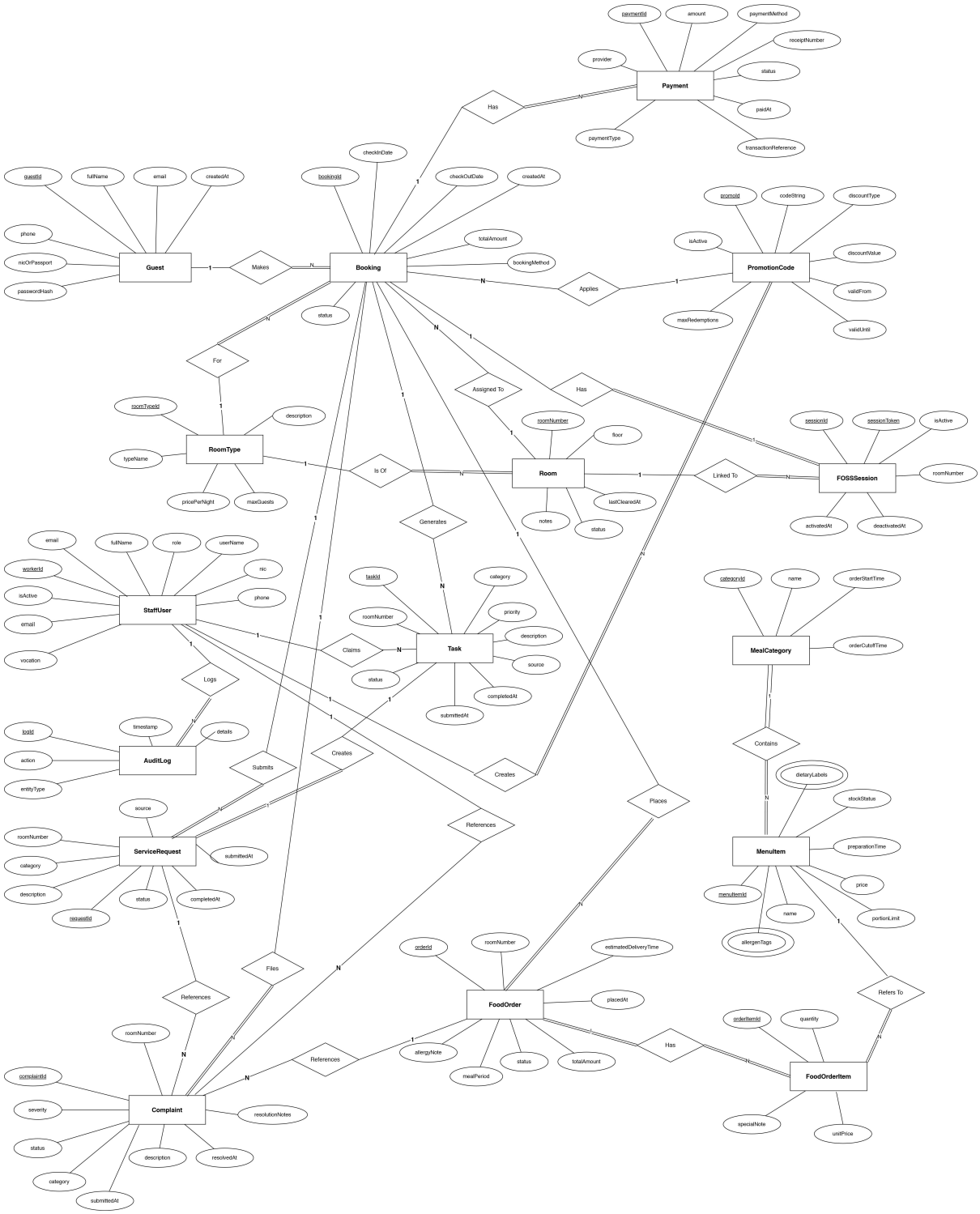


Figure 2 - ER Diagram

2.2 Normalization explanation

It is demonstrated in this section that the Forever Hotel database schema satisfies Third Normal Form (3NF) and Boyce-Codd Normal Form (BCNF). The sixteen entities identified in the ER Diagram are covered in the analysis (Section 2.1) and are grounded in the data requirements defined in SRS Section 13.

2.2.1 Normal Forms Applied

Table 4 - Normal Forms

Normal Form	Requirement	Status in Forever Hotel
1NF- First Normal Form	Every attribute is atomic (single-valued); no repeating groups; every row has a primary key.	Satisfied - All 16 entities use atomic attributes and a single-column UUID PK. Repeating groups (food orders, tasks, complaints, service requests) are extracted into separate entities.
2NF - Second Normal Form	In 1NF; every non-key attribute is fully dependent on the entire primary key (no partial dependencies).	Satisfied - All entities use single-column PKs, making partial dependencies structurally impossible. The one M:N relationship (FoodOrder–MenuItem) is resolved using a separate junction entity FoodOrderItem, eliminating repeating groups and ensuring full functional dependency.
3NF - Third Normal Form	In 2NF; no non-key attribute depends on another non-key attribute (no transitive dependencies).	Satisfied - Six transitive dependencies identified and resolved by extracting separate entities (RoomType, MealCategory, Guest, StaffUser, PromotionCode, Payment). One controlled denormalization (roomNumber) is explicitly justified.

BCNF - Boyce-Codd Normal Form	For every functional dependency X → Y, X must be a superkey.	Satisfied - Since all non-key attributes depend on the primary key and all candidate keys are superkeys, BCNF is satisfied. Candidate keys (email, username, codeString, sessionToken) are all superkeys.

2.2.2 From Unnormalized Form (UNF) to 1NF

Before normalization, a hypothetical flat STAY_RECORD would combine all hotel stay data - guest details, room information, staff data, orders, tasks, and complaints - into one wide row with nested repeating groups:

STAY_RECORD(bookingId, guestName, guestEmail, guestPhone,
roomNumber, roomTypeName, pricePerNight,
staffName, staffRole,
{orderId, mealPeriod, {itemName, qty, price}, allergyNote},
{requestId, requestCategory, requestStatus},
{taskId, category, priority, workerName},
{complaintId, severity, description})

1NF is violated in five ways by this UNF record. Each violation is shown in the table below, and the entity the ER diagram creates to resolve it.

Table 5 - Normal Forms Applied

1NF Violation	**Problem**	**ER Diagram Resolution**	**SRS Ref.**
Repeating food orders (each with repeating items)	A booking row contains a list of orders; each order contains a list of items. (nested repetition)	Extracted into two entities: FoodOrder (order-level data) and FoodOrderItem (item-level data). Each item is	SRS 13.1.3 GA04–GA09

		stored as a separate row linked via FK (orderId).	
Repeating service requests	Multiple requests (towels, cleaning, maintenance) nested inside one booking row.	ServiceRequest entity (PK: requestId). Booking Submits ServiceRequest (1:N).	SRS 13.1.7 GA10–GA12
Repeating tasks	Tasks from multiple sources (guest app, checkout, KMS) nested in one row.	Task entity (PK: taskId). Three separate relationships generate tasks: Booking→Task, ServiceRequest→Task, KMS delivery.	SRS 13.1.4 WM01, FD10
Repeating complaints	Multiple complaints per stay, each with category, severity, and optional references.	Complaint entity (PK: complaintId). Booking Files Complaint (1:N). Three nullable FK references for order/request/worker.	SRS 13.1.9 GA13–GA15
Guest & staff data repeated across rows	guestName, guestEmail, workerName, workerRole duplicated in every booking or task row.	Guest entity (PK: guestId) and StaffUser entity (PK: workerId) are standalone. Booking and Task reference them via FK.	SRS 13.1.2 SRS 13.1.8
Non-atomic attributes in MenuItem (dietaryLabels, allergenTags)	Attributes can hold multiple values (e.g., multiple dietary labels or allergen tags per menu item). In the ER diagram, these are correctly modelled as	Modelled as double-ellipse (multi-valued) attributes on MenuItem in the ER diagram. No separate entity is needed at the ER level; resolution into separate rows occurs at the	SRS 13.1.6

	double-ellipse multi-valued attributes on MenuItem, which is valid ER notation. They will be implemented as separate lookup tables at the physical schema stage. Note: these are distinct from allergyNote on FoodOrder (a free-text atomic string per order) and specialNote on FoodOrderItem (a free-text atomic string per line item) — both of which are already atomic and require no further resolution.	relational/physical schema stage.	

In the initial design, food order items were stored as a JSONB array inside the FoodOrder entity. This approach violates 1NF because it stores multiple values within a single attribute.

To achieve proper normalization, the design is updated by introducing a separate FoodOrderItem entity. Each menu item in an order is stored as an individual row with attributes such as quantity and unitPrice, and linked to FoodOrder via a foreign key.

This eliminates nested repetition and ensures that all attributes are atomic and compliant with 1NF.

After these extractions, every table contains only atomic, single-valued columns and is uniquely identified by its primary key. All entities use atomic attributes. The multi-valued attributes dietaryLabels and allergenTags on MenuItem are modelled as double-ellipse attributes in the ER diagram, correctly indicating they hold multiple values. These will be resolved into separate lookup tables at the physical schema stage. These multi-valued attributes are distinct from allergyNote on FoodOrder (a free-text atomic note per order) and specialNote on FoodOrderItem (a free-text atomic note per line item), both of which are single-valued and already satisfy 1NF without further decomposition.

2.2.3 Second Normal Form (2NF)

2NF violations only arise when a table has a composite (multi-column) primary key. In the Forever Hotel schema, all 16 entities use a single-column UUID primary key. Partial dependencies are therefore structurally impossible across the entire schema.

The one situation that would produce a composite key is the M:N relationship between FoodOrder and MenuItem (the Includes relationship). A standard junction table would have the form:

FoodOrderItem(orderItemId PK, orderId FK, menuItemId FK, name, quantity, unitPrice)
UNIQUE: (orderId, menuItemId)

Checking 2NF: quantity and unitPrice both depend on the full composite key (orderId, menuItemId) - fully functionally dependent on both columns, with no partial dependency on either alone. This design is implemented as the FoodOrderItem entity (PK: orderItemId; FKs: orderId → FoodOrder, menuItemId → MenuItem), which resolves the M:N relationship into two clean 1:N relationships. Each FoodOrderItem row captures the name and unitPrice at the time of ordering, which also prevents price-change anomalies in historical order records. All other ER diagram relationships are 1:N, implemented via FK on the many-side entity — no composite keys, no 2NF risk.

2.2.4 Third Normal Form (3NF) Transitive Dependency Analysis

Six transitive dependencies were identified by examining the SRS data requirements and ER diagram. Each was resolved by extracting a separate entity. One instance of intentional denormalization is explicitly justified.

Table 6 - Third Normal Form (3NF) Transitive Dependency Analysis

	Transitive Dependency	**Chain**	**Resolution**	**SRS Ref.**
1	Room type attributes inside Room	roomNumber → roomTypeId → typeName, pricePerNight, maxGuests	Extracted to RoomType entity (PK: roomTypeId). Room holds only physical properties. Is Of (RoomType 1:N Room).	SRS 13.1.5 HW03
2	Ordering time window inside MenuItem	menuItemId → categoryId → orderStartTime, orderCutoffTime	Extracted to MealCategory entity (PK: categoryId). Contains (MealCategory 1:N MenuItem).	SRS 13.1.6 KMM01
3	Guest details inside Booking	bookingId → guestId → fullName, email, phone, nicOrPassport	Extracted to Guest entity (PK: guestId). Makes (Guest 1:N Booking).	SRS 13.1.2 HW06–HW14
4	Staff details inside Task	taskId → assignedWorkerId → fullName, role, email, vocation	Extracted to StaffUser entity (PK: workerId). Claims (StaffUser 1:N Task).	SRS 13.1.8 WM03–WM10

5	Promotion attributes inside Booking	bookingId → promoId → discountType, discountValue, validFrom, validUntil	Extracted to PromotionCode entity (PK: promoId). Applies (Booking N:1 PromotionCode).	SRS MD05 HW10
6	Payment attributes inside Booking	bookingId → paymentMethod, amount, paymentStatus, paidAt	Extracted to Payment entity (PK: paymentId). Booking Makes Payment (1:N). Payment references Booking via FK.	SRS 13.1.1

Controlled Denormalization - roomNumber

Situation: roomNumber appears as a denormalized attribute in FoodOrder, ServiceRequest, Task, and Complaint. These entities already reference Booking (via bookingId), so roomNumber could theoretically be derived through a join.

Justification: The SRS explicitly documents this as an intentional performance optimization (SRS Section 13.1.3: 'denormalised for kitchen display performance'). In real-time subsystems - the KMS Kanban board (SRS KM03), the WKMS task queue (SRS WM02), and the FDS escalation panel (SRS FD17) - retrieving roomNumber without joining through Booking is required to meet NFR-01 (95% of API responses within 500 ms). Because roomNumber does not change after check-in, there is no update anomaly risk. This is a recognized and documented controlled redundancy.

2.2.5 Boyce-Codd Normal Form (BCNF)

BCNF requires that for every functional dependency X → Y, X must be a superkey. All 16 entities use single-column UUID primary keys, so every functional dependency trivially satisfies this. Three entities have additional candidate keys that also require verification:

Table 7 - Boyce-Codd Normal Form (BCNF)

Entity	Candidate Keys (beside PK)	BCNF Check
Guest	email (UNIQUE)	email → all attributes. email is a superkey.
StaffUser	username (UNIQUE), email (UNIQUE)	Both candidate keys are superkeys. All FDs have a superkey on the left.
PromotionCode	codeString (UNIQUE)	codeString → all attributes. codeString is a superkey.
FOSSSession	bookingId (UNIQUE), sessionToken (UNIQUE)	sessionToken is a superkey and the only candidate key beside the PK. roomNumber is a FK to Room (not a candidate key — a room may host multiple sessions over time). FOSSSession is connected to Room via “Linked To” and to Booking via “Has” (both relationships visible in ER diagram). All FDs have a superkey on the left.

No BCNF violations exist in the Forever Hotel schema.

2.2.6 Anomaly Elimination

Three classical database anomalies are eliminated through normalization. Key examples from the Forever Hotel schema are listed below.

Table 8 - Anomaly Elimination

Anomaly Type	Definition	Forever Hotel Example
Insertion Anomaly	Cannot insert data without unrelated data being present.	A new RoomType or MenuItem can be added before any booking or order references it. A PromotionCode can be created (SRS MD05) before any guest applies it.
Update Anomaly	Changing one fact requires modifying multiple rows.	Updating a room type's pricePerNight requires changing only one RoomType row — not every Room and Booking row. Correcting a MenuItem's allergenTags updates one row; all FOSS views reflect it immediately.
Deletion Anomaly	Deleting one record accidentally destroys unrelated information.	Cancelling a Booking does not delete the Guest record (SRS HW14, retention policy SRS 13.2). Deactivating a StaffUser (SRS MD16b) does not erase their Task history or AuditLog entries.

2.2.7 Normalized Schema Summary

The normalization status is summarized in the table below and primary key of all 16 entities. Entities marked * contain the roomNumber controlled denormalization described in Section 2.2.4.

Table 9 - Normalized Schema Summary

Entity	Primary Key	1NF	2NF	3NF/BCNF	Key Notes
Guest	guestId	✓	✓	✓	Candidate key: email

Entity	Primary Key	1NF	2NF	3NF/BCNF	Key Notes
Booking	bookingId	✓	✓	✓	FK to Guest, RoomType, Room, Promotion Code, FOSSSession (“Has” relationship); payment data moved to Payment entity
Payment	paymentId	✓	✓	✓	Resolves Transitive Dep. #6; FK to Booking; stores paymentMethod, amount, paymentStatus, paidAt
RoomType	roomTypeId	✓	✓	✓	Resolves Transitive Dep. #1
Room	roomNumber	✓	✓	✓	FK to RoomType; physical properties only
StaffUser	workerId	✓	✓	✓	Candidate keys: username, email; all roles
FoodOrder	orderId	✓	✓	✓ *	FK to MenuItem via FoodOrderItem; roomNumber denorm.
FoodOrderItem	orderItemId	✓	✓	✓	Junction entity resolving FoodOrder–MenuItem M:N; FKs to orderId and menuItemId; stores quantity and unitPrice at order time
MealCategory	categoryId	✓	✓	✓	Resolves Transitive Dep. #2
MenuItem	menuItemId	✓	✓	✓	FK to MealCategory; dietaryLabels and allergenTags are multi-valued attributes (double ellipse in ER diagram) on MenuItem only — distinct from atomic allergyNote (FoodOrder) and specialNote (FoodOrderItem); resolved to separate lookup tables at physical schema stage

Entity	Primary Key	1NF	2NF	3NF/BCNF	Key Notes
ServiceRequest	requestId	✓	✓	✓ *	1:1 with Task; roomNumber denorm.
Task	taskId	✓	✓	✓ *	Nullable FK to StaffUser; roomNumber denorm.
Complaint	complaintId	✓	✓	✓ *	3 nullable FK refs; roomNumber denorm.
PromotionCode	promoId	✓	✓	✓	Resolves Transitive Dep. #5; candidate: codeString
FOSSSession	sessionId	✓	✓	✓	Candidate key: sessionToken; roomNumber is a FK to Room (not a candidate key); linked to Room via "Linked To" and to Booking via "Has" (both relationships shown in ER diagram)
AuditLog	logId	✓	✓	✓	Append-only; 3-year retention (SRS 13.2); FK to StaffUser via "Logs" relationship (StaffUser 1:N AuditLog); entityType attribute records the audited entity type; note: the "Creates" relationship in the ER diagram runs from StaffUser to ServiceRequest, not from AuditLog

* Intentional controlled denormalization — roomNumber stored for real-time display performance (SRS Sections 13.1.3, 13.1.4, 13.1.7, 13.1.9; NFR-01).

The Forever Hotel database schema achieves full 3NF and BCNF compliance across all 16 entities. All six identified transitive dependencies have been resolved through entity separation, as reflected in the ER Diagram (Section 2.1). The one controlled denormalization is explicitly documented and justified in the SRS.

CHAPTER 3: CLASS DIAGRAM

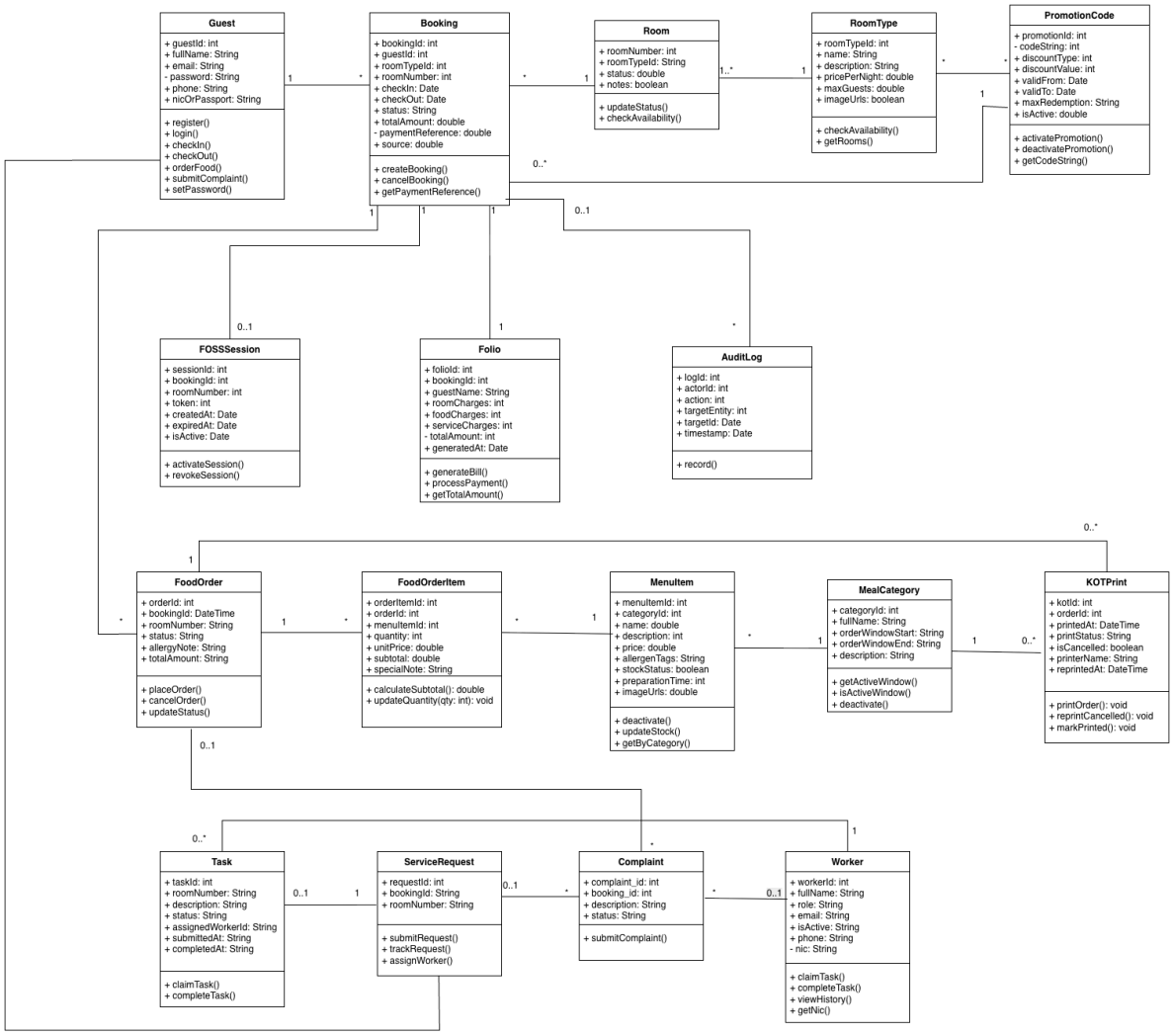


Figure 3 - Class Diagram

CHAPTER 4: SEQUENCE DIAGRAMS

4.1 Online Room Booking

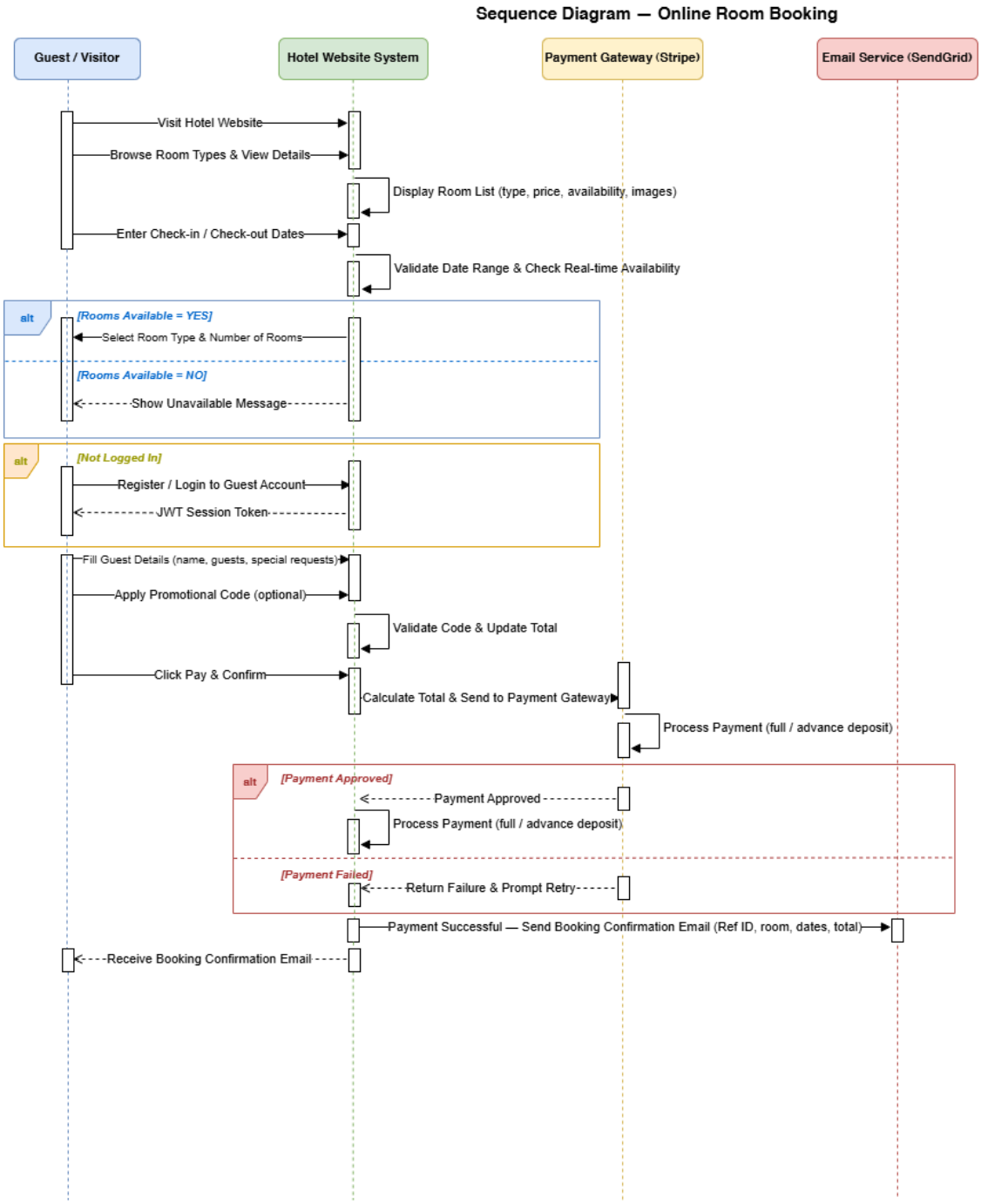


Figure 4 - Online Room Booking

4.2 Guest Check-in

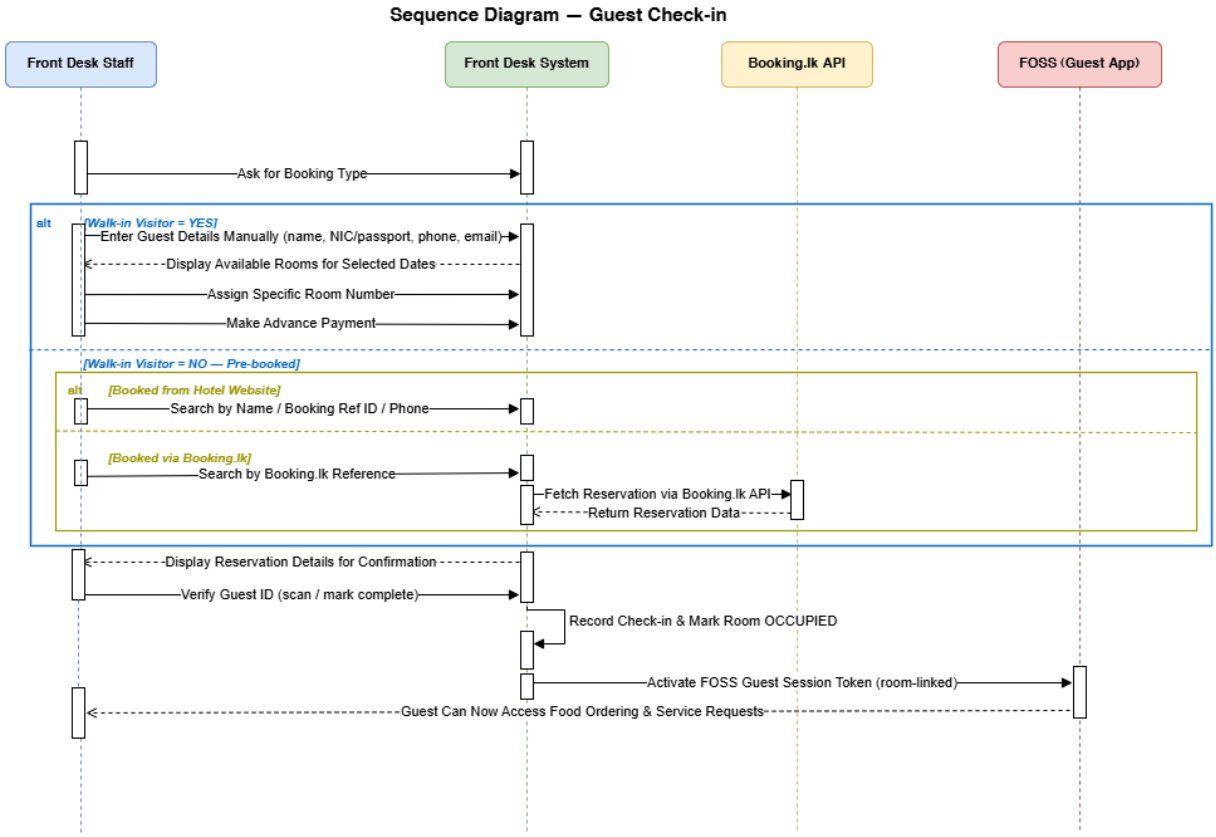


Figure 5 - Guest-Check-In

4.3 Guest Check-out

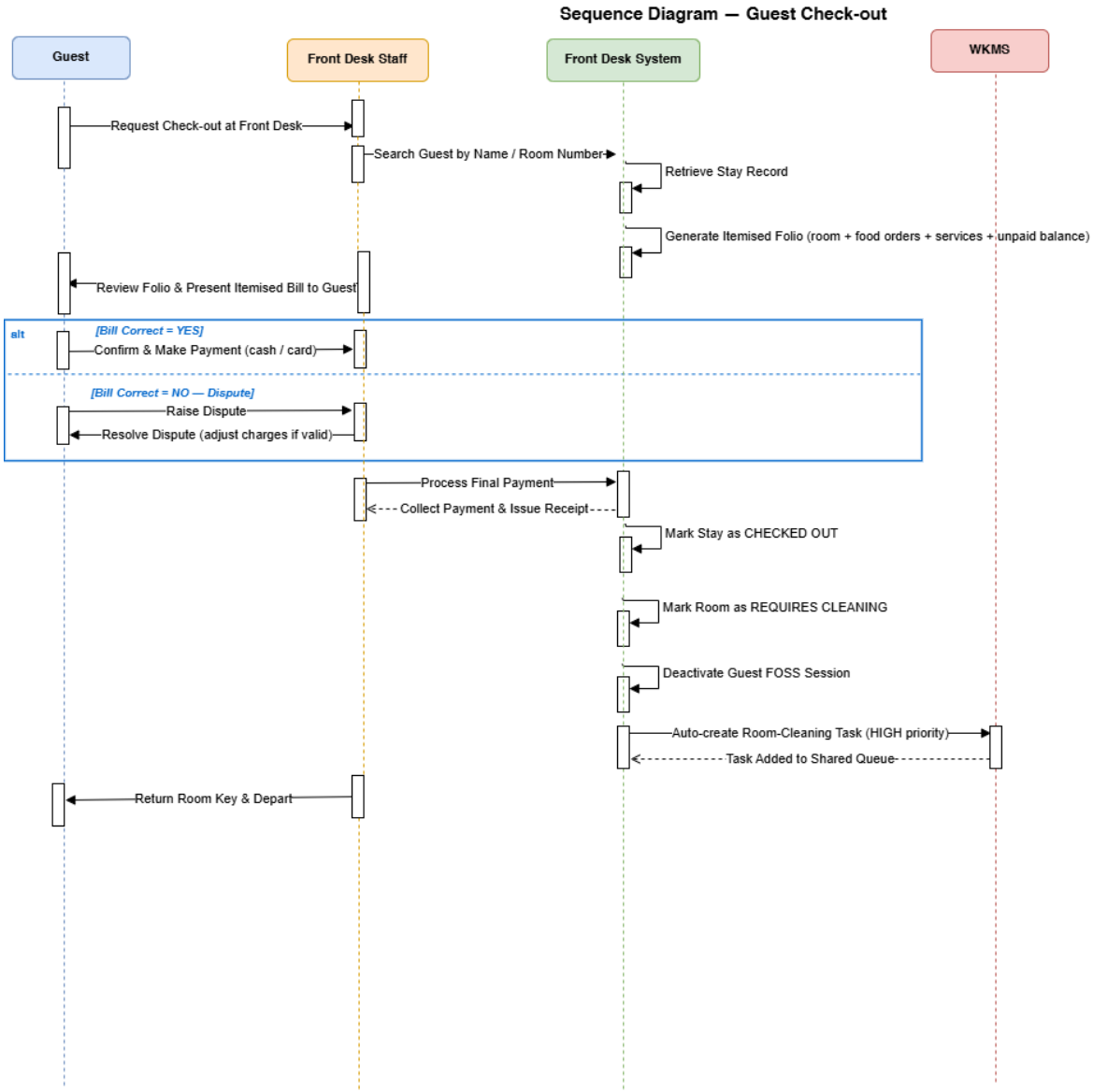


Figure 6 - Guest-Check-Out

4.4 Worker Registration

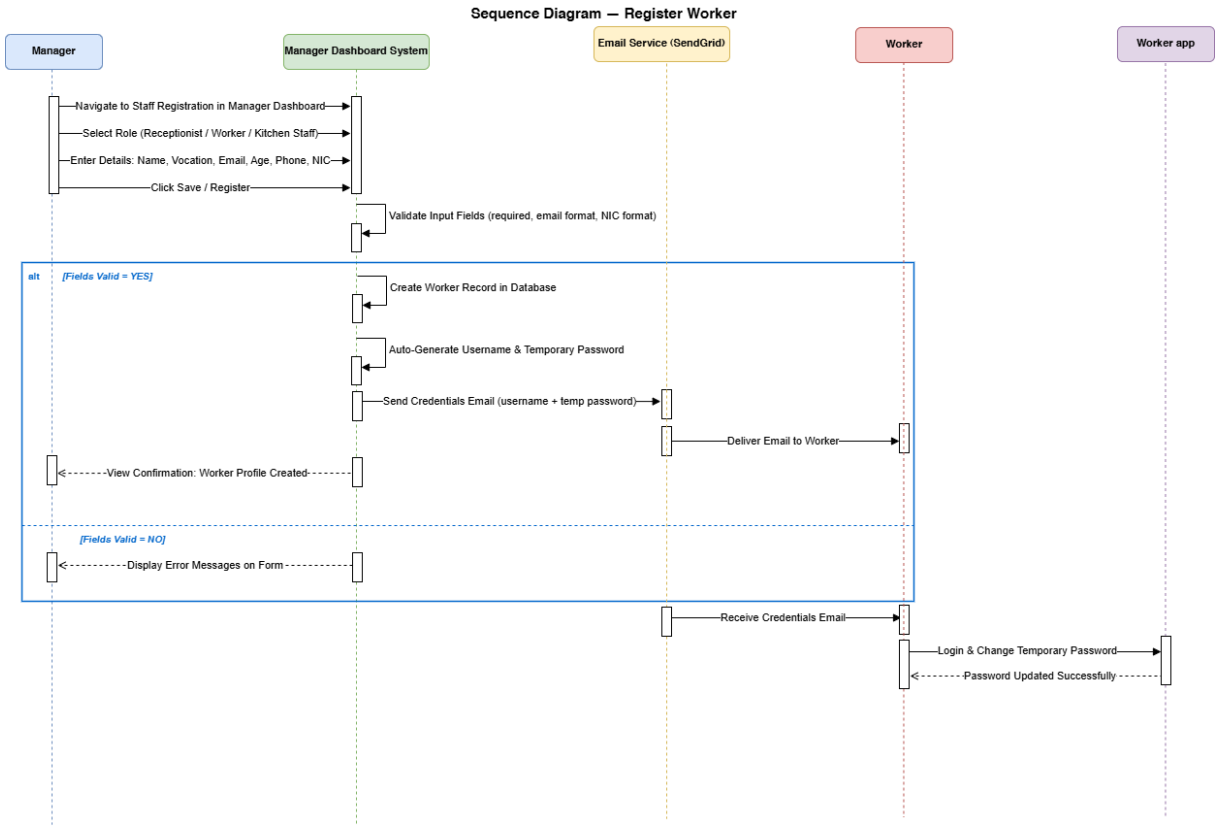


Figure 7 - Worker Registration

4.5 Worker Complaint Management

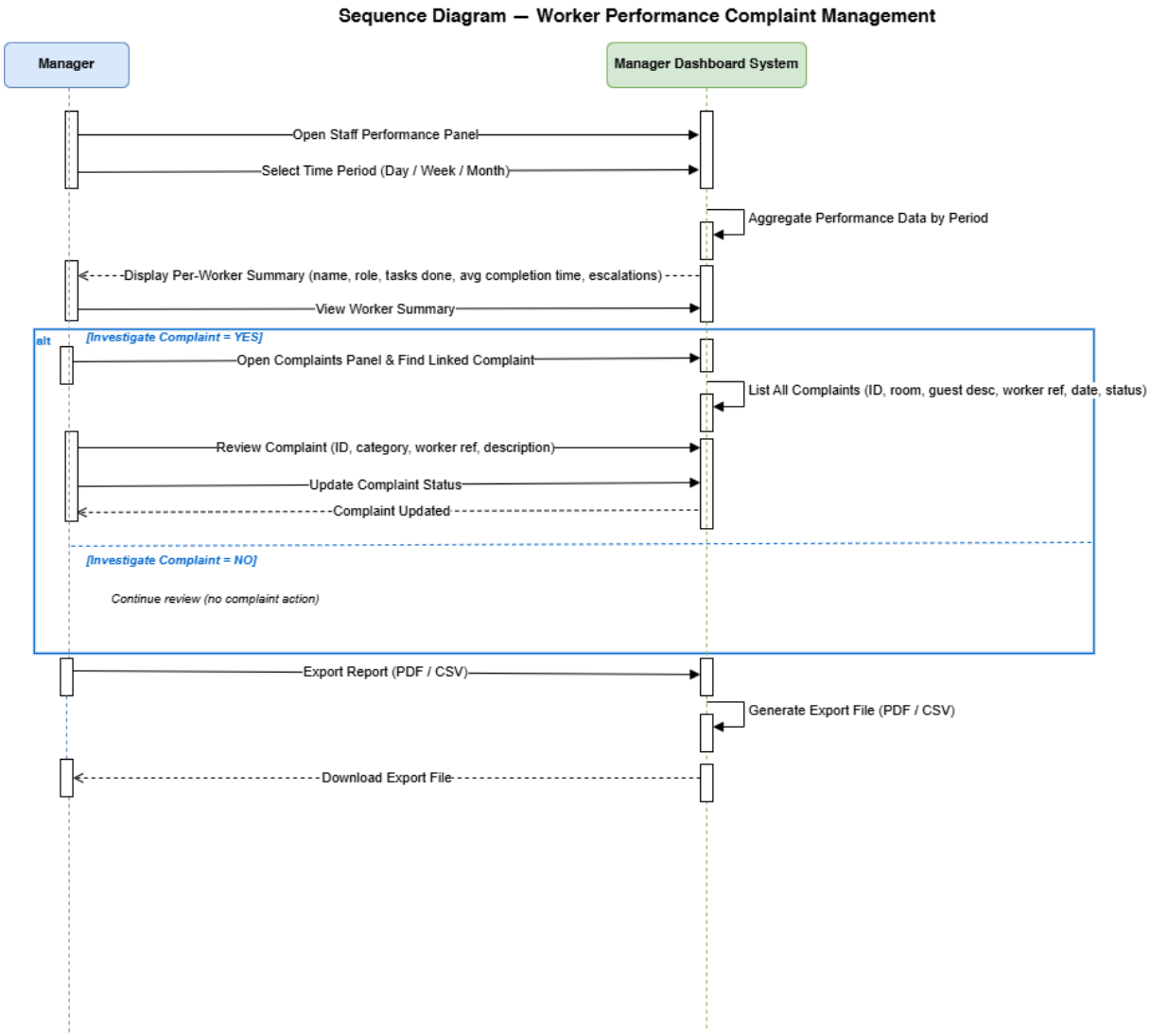


Figure 8 - Worker Complaint Management

4.6 Manager Analytics

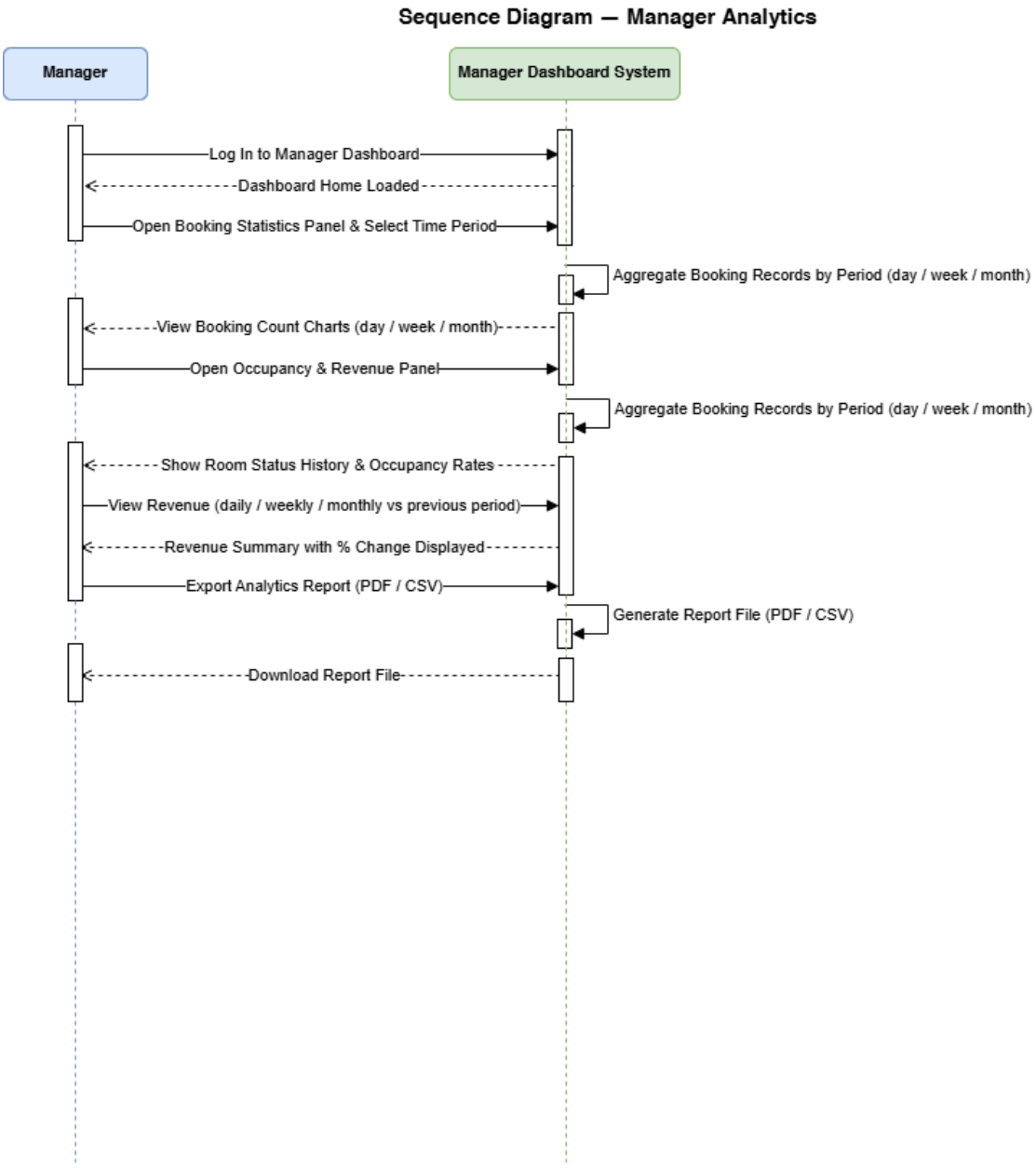


Figure 9 - Manager Analytics

4.7 Manage Promotions

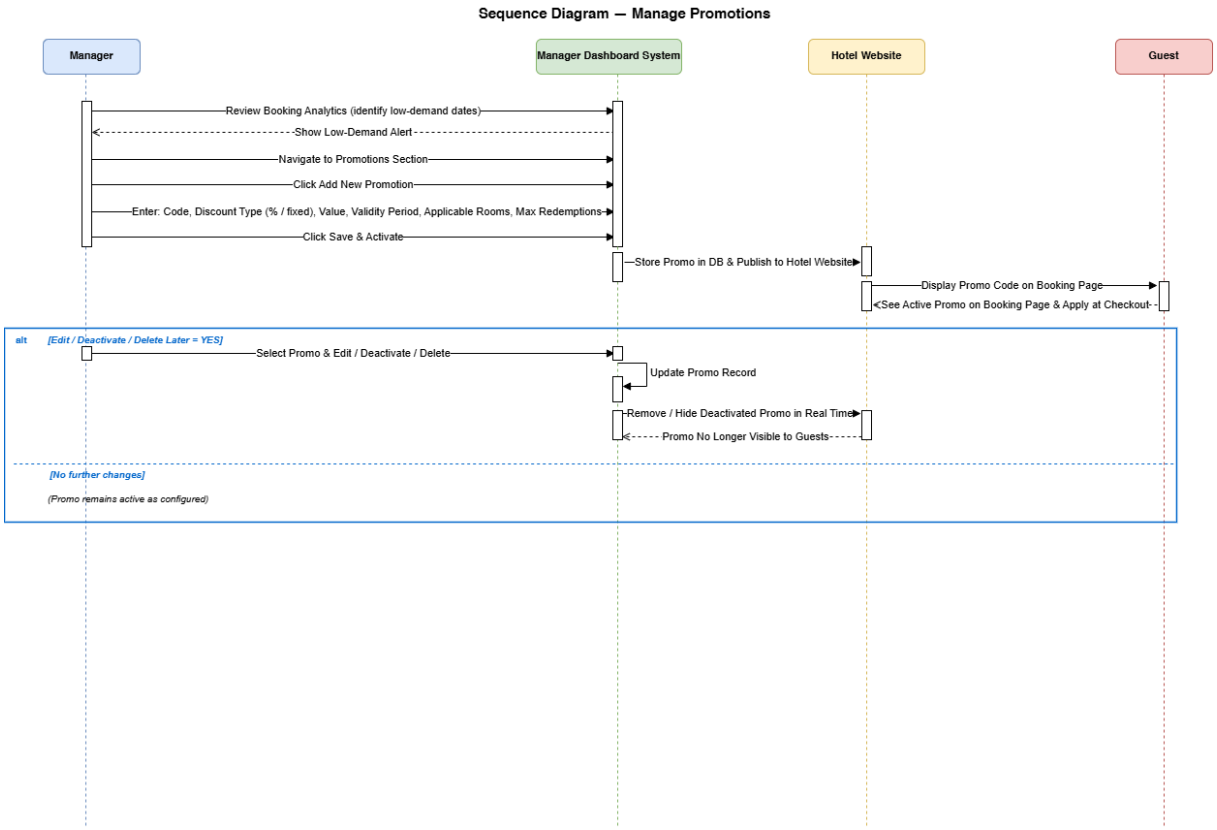


Figure 10 - Manage Promotions

4.8 Order Food

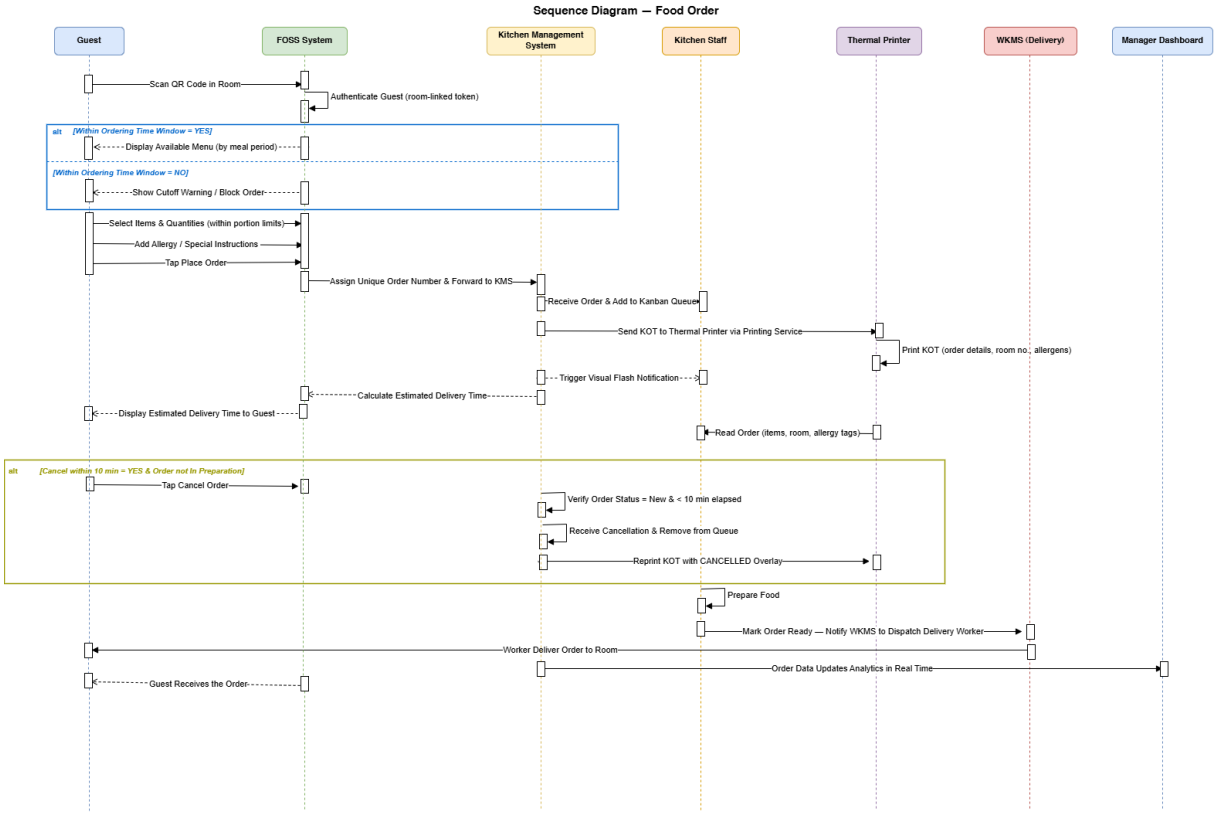


Figure 11 - Order Food

4.9 Request Service

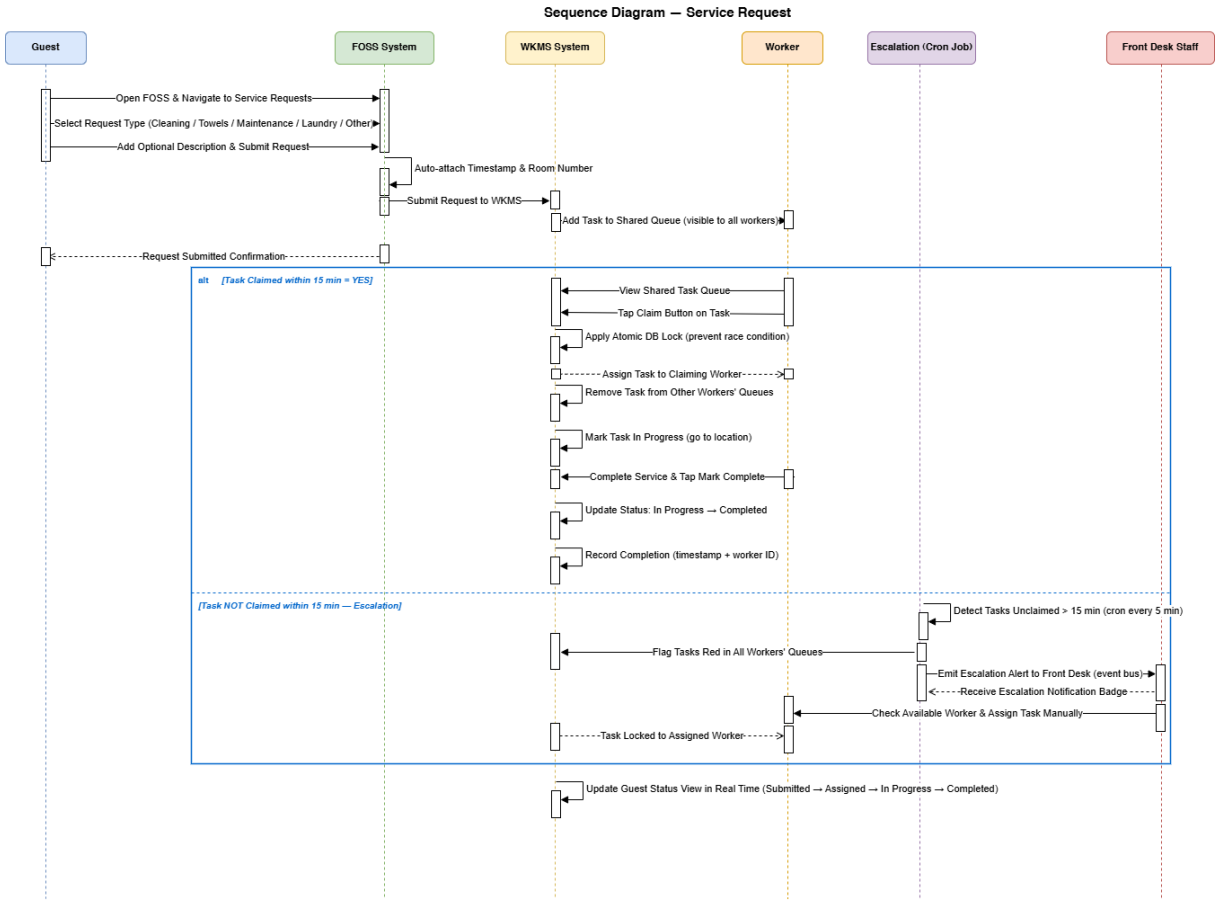


Figure 12 - Request Service

4.10 Worker Task Claim

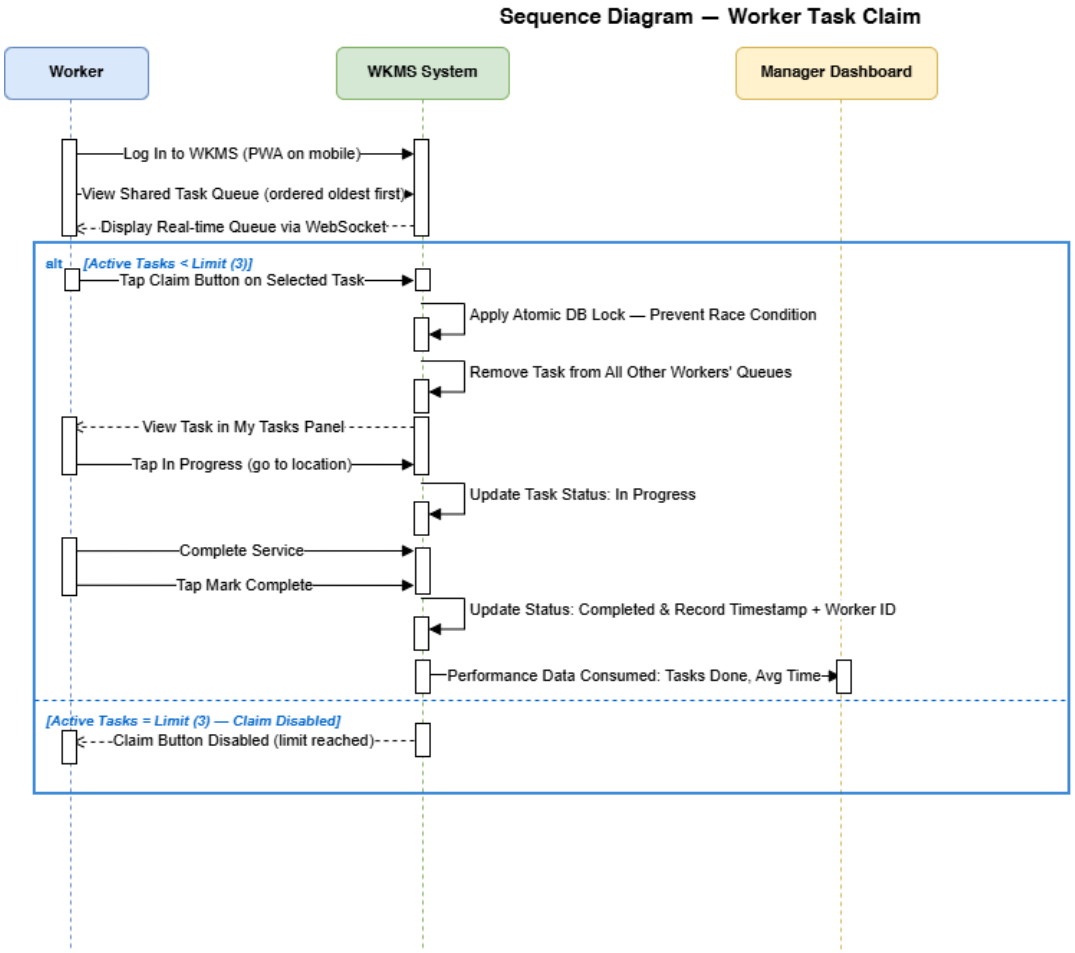


Figure 13 - Worker Task Claim

CHAPTER 5: UI/UX DESIGN

5.1 Wireframes or screen mock-ups for key user interfaces

The user interface of the system has been designed using Figma.

All major subsystems are included in the mockups, such as:

1) Hotel Website
2) Manager Dashboard
3) Front Desk System
4) Guest App
5) Kitchen Management System
6) Worker Management System

The screen mock-ups for key user interfaces can be accessed using the following link:

Figma Design [14]

5.2 User Flow and Navigation Diagrams

5.2.1 Hotel Website (HW) - User Flows

Two user roles are served by the Hotel Website: anonymous Visitor and authenticated Registered Guest. Four primary flows are defined as covering account registration, login, room booking, and booking self-service.

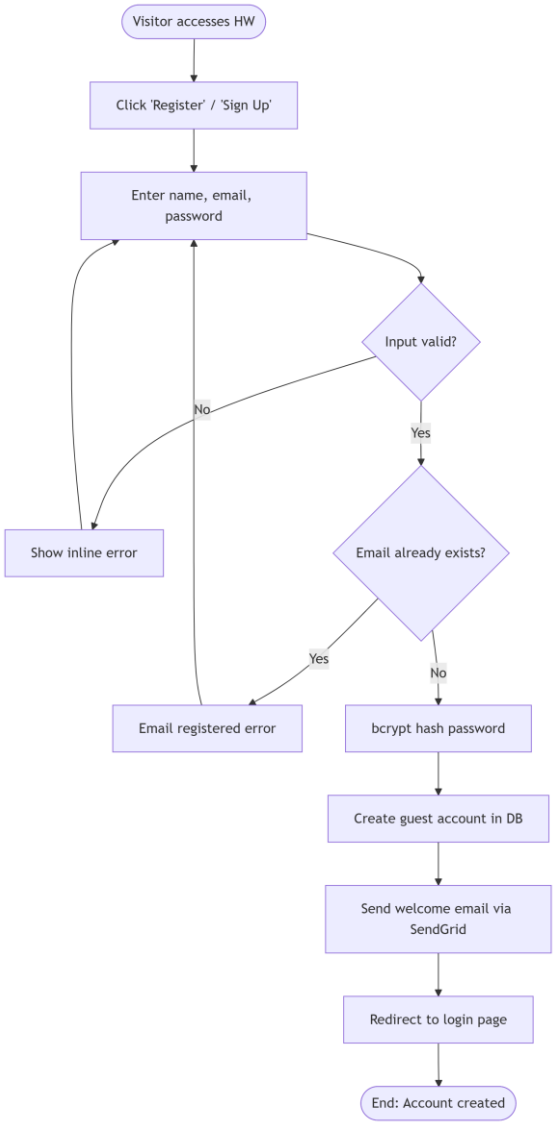


Figure 14 - HW-01: Guest Account Registration Flow

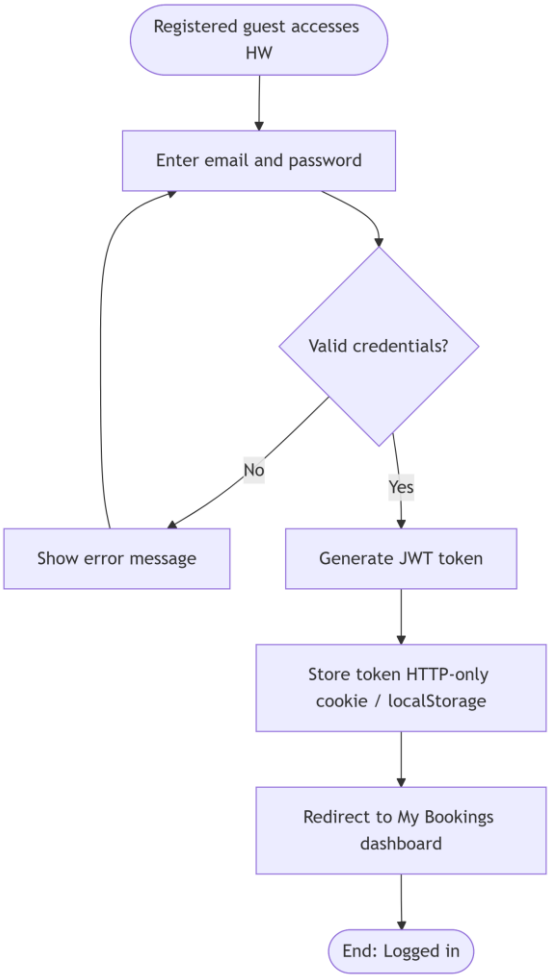


Figure 15 - HW-02: Guest Login Flow

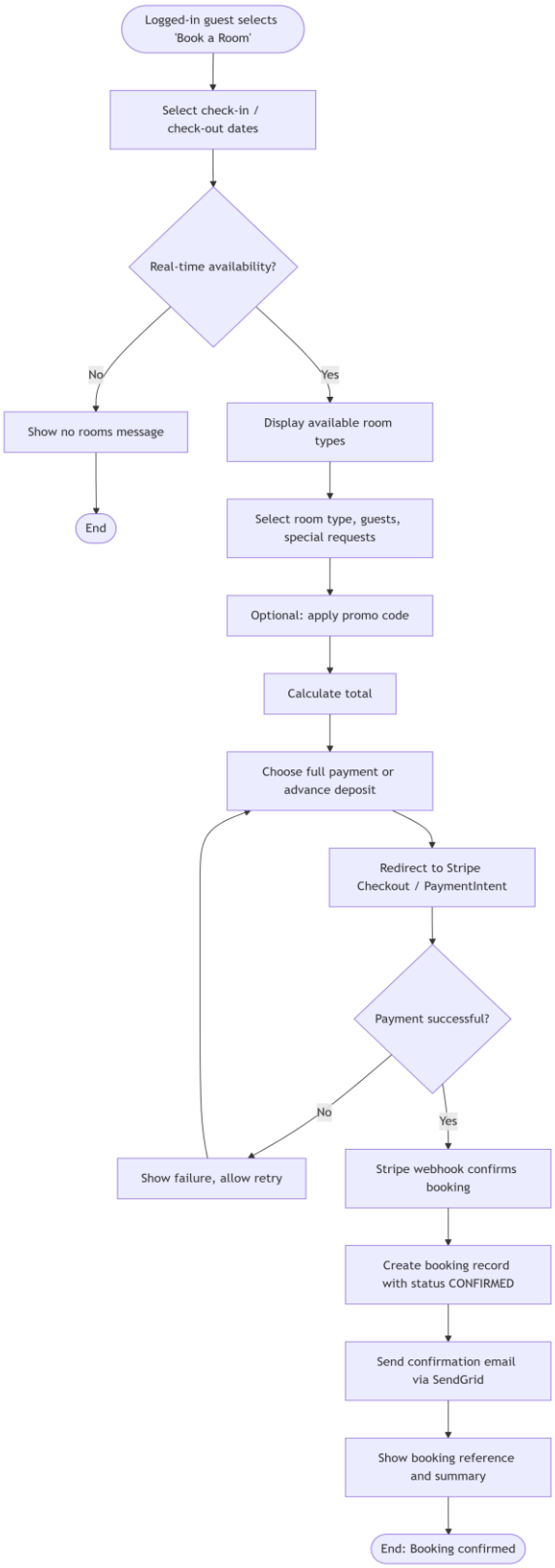


Figure 16 - HW-03: Online Room Booking Flow

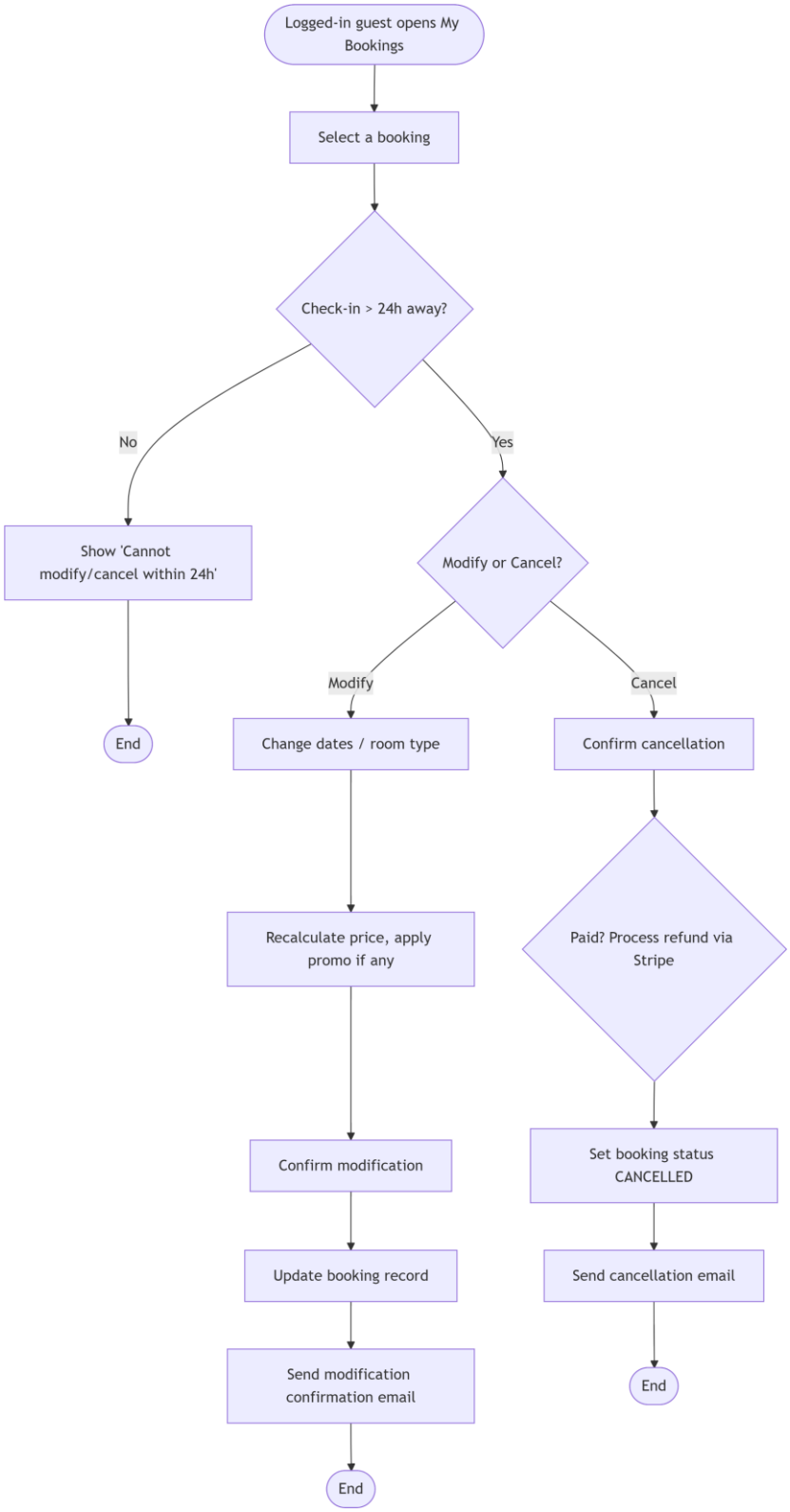


Figure 17 - HW-04: Booking Modification and Cancellation Flow

5.2.2 Manager Analytical Dashboard (MAD) - User Flows

The Hotel Manager role is served by the Manager Dashboard. Six flows are defined covering analytics review, staff account management, password administration, promotion management, complaint resolution, and export report.

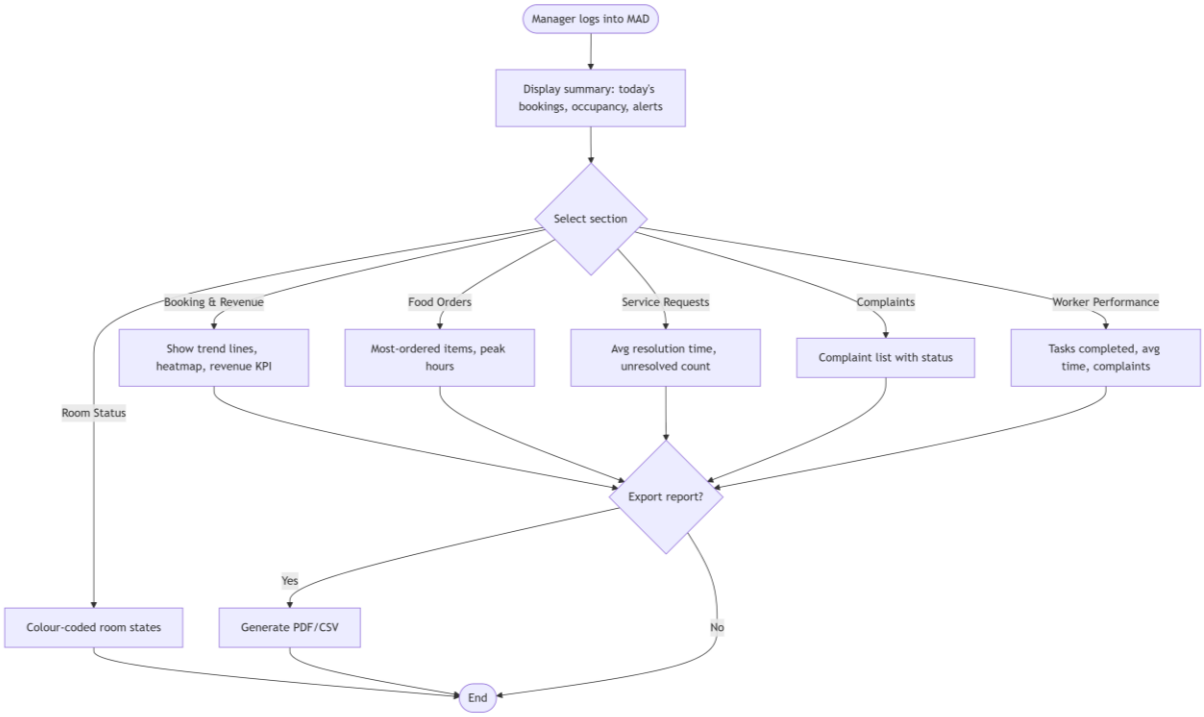


Figure 18 - MAD-05: Analytics and Operations Overview Flow

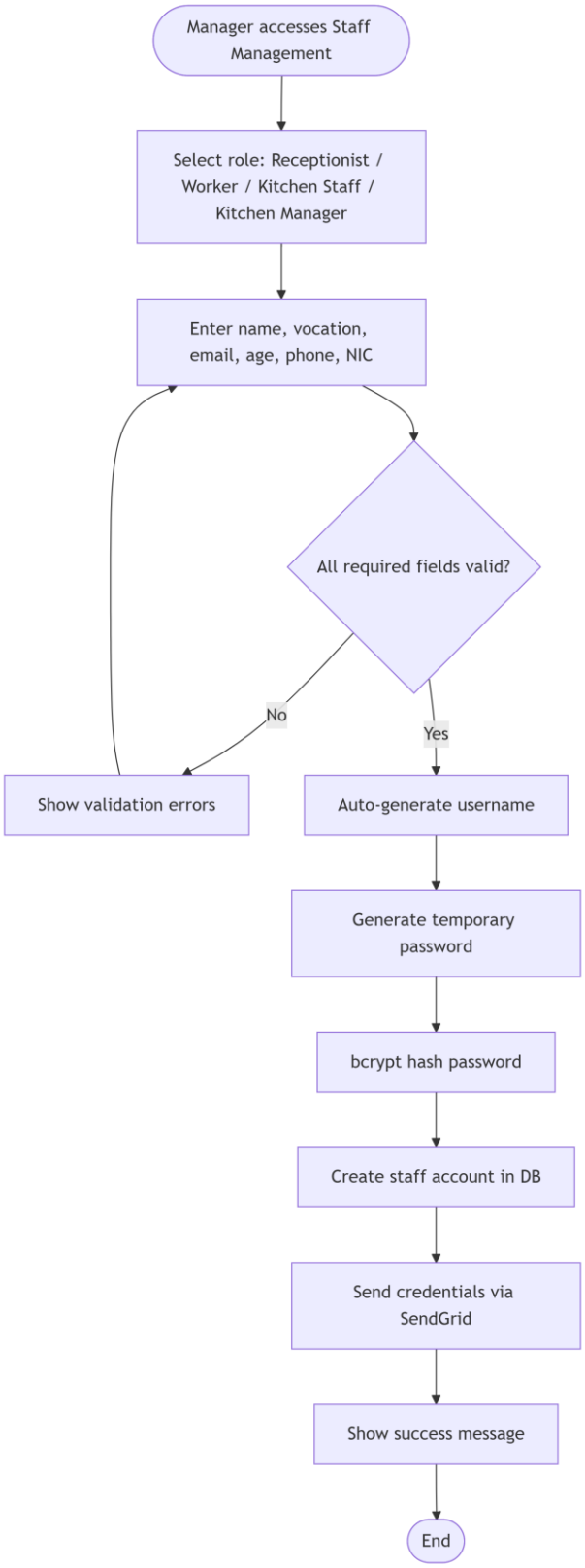


Figure 19 - MAD-06: Staff Account Creation and Credential Delivery Flow

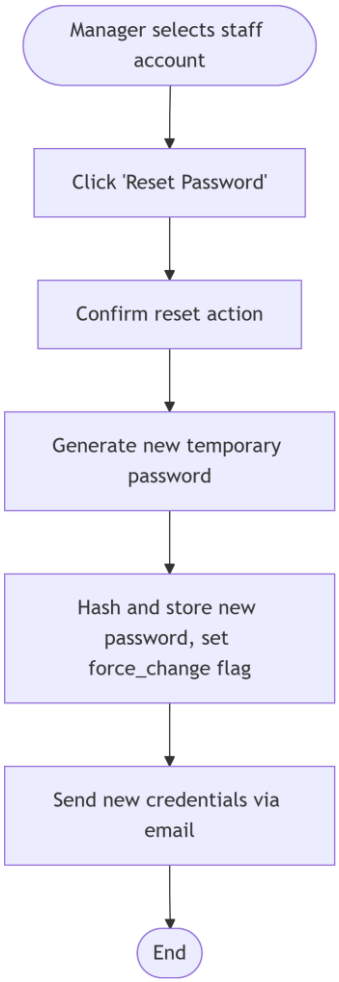


Figure 20 - MAD-07: Staff Password Reset Flow

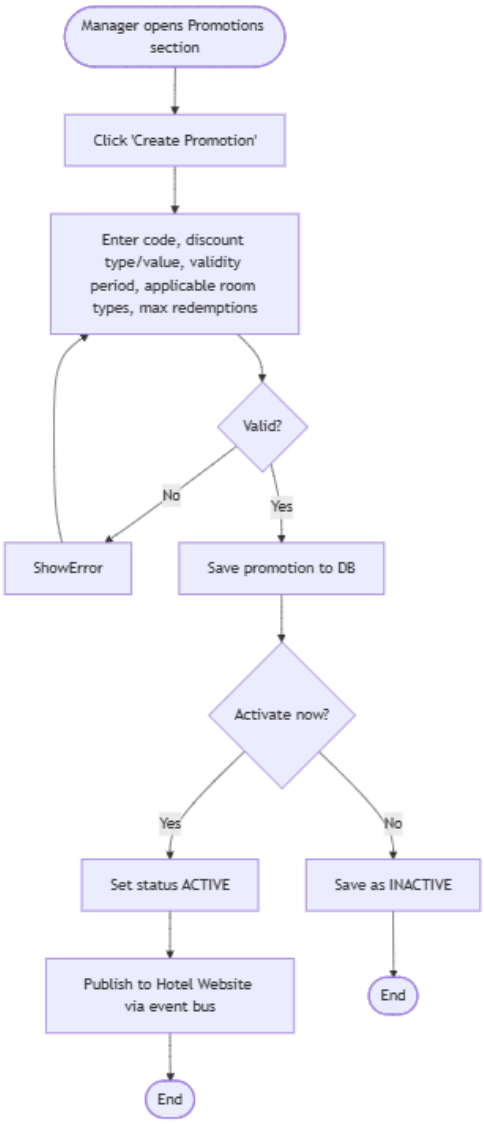


Figure 21 - MAD-08: Promotion Creation and Publish Flow

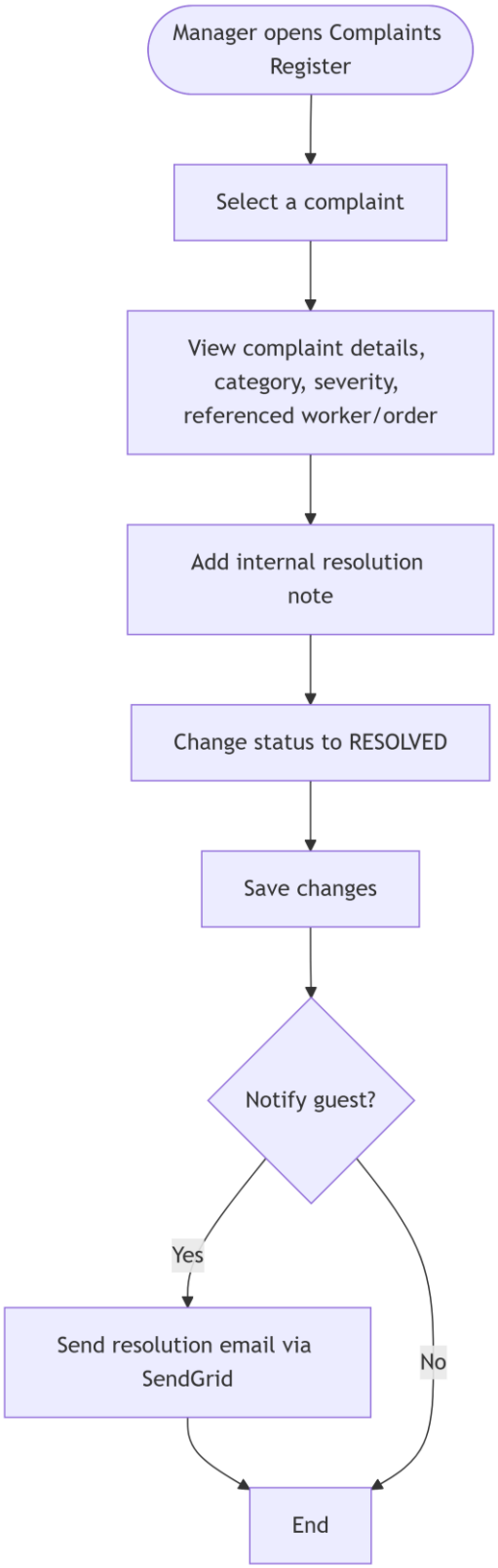


Figure 22- MAD-09: Complaint Resolution Flow

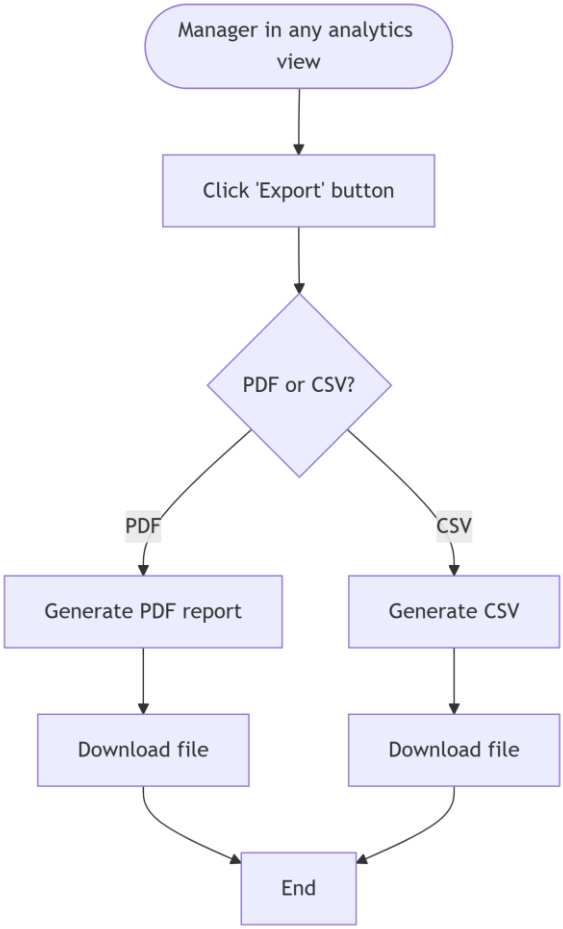


Figure 23 - MAD-10: Report Export Flow

5.2.3 Front Desk System (FDS) - User Flows

The Receptionist role is served by the Front Desk System. Five flows are defined covering guest check-in, check-out, room reassignment, escalation management, and service request creation on behalf of guests.

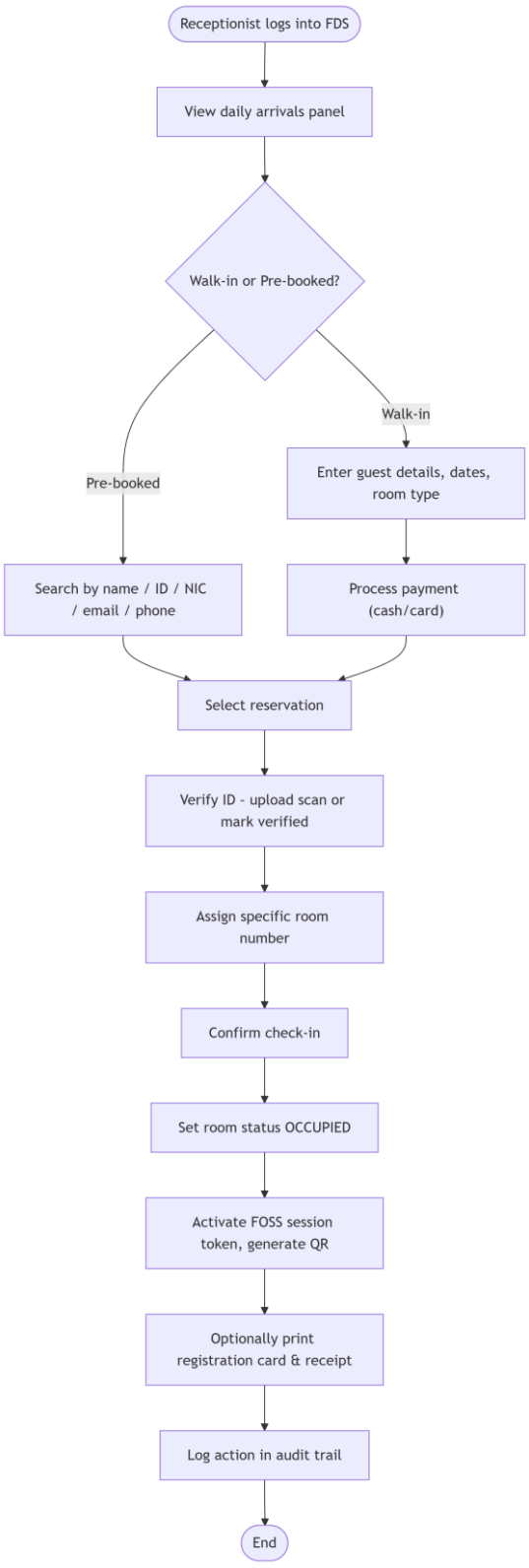


Figure 24 - FDS-11: Guest Check-In (Pre-Booked and Walk-In) Flow

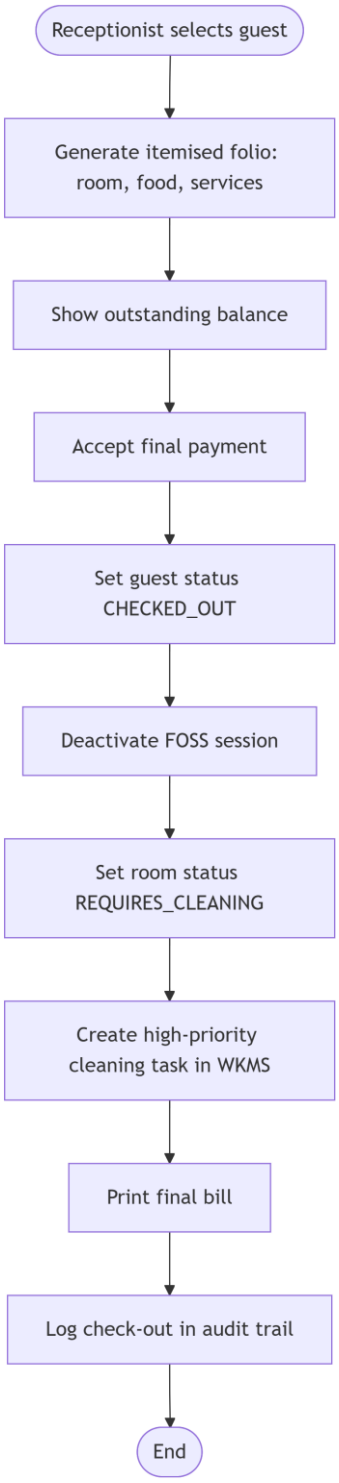


Figure 25 - FDS-12: Guest Check-Out Flow

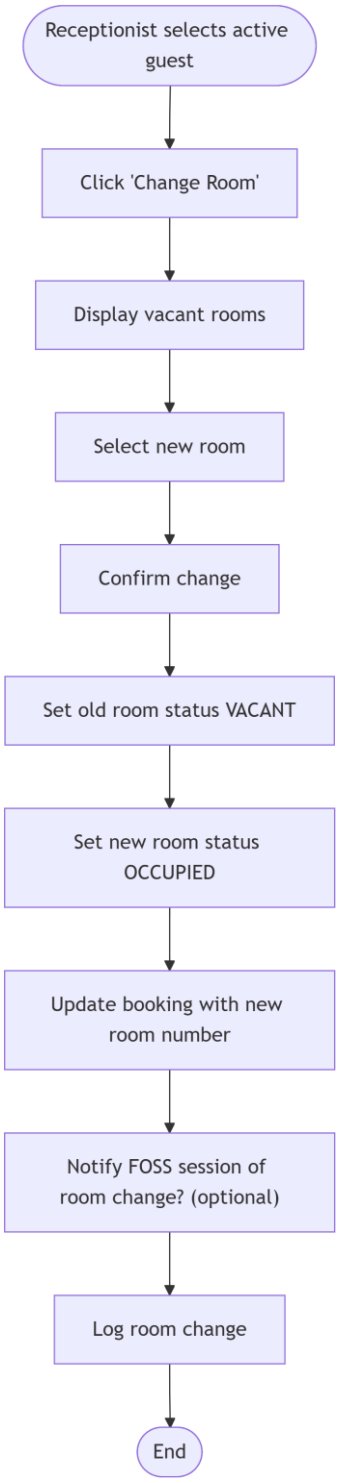


Figure 26 - FDS-13: Room Change and Reassignment Flow

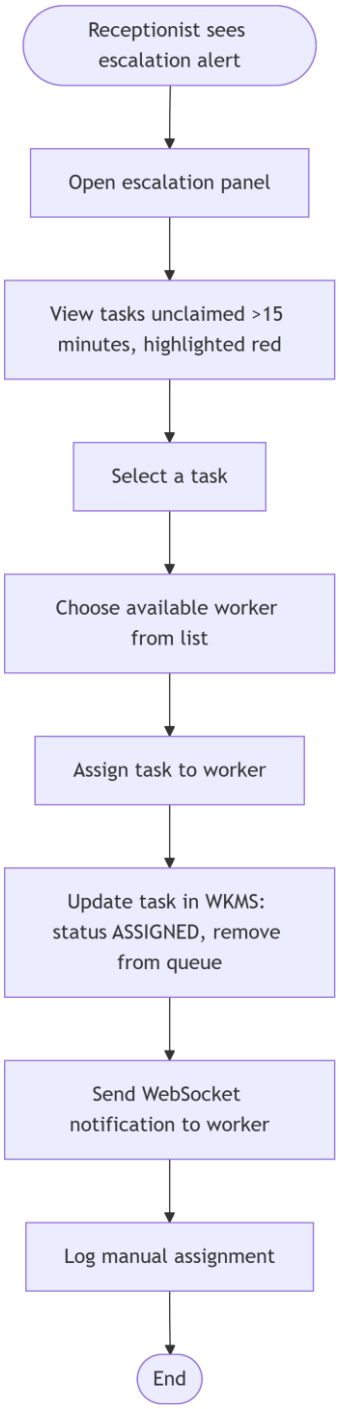


Figure 27 - FDS-14: Manual Escalation Task Assignment Flow

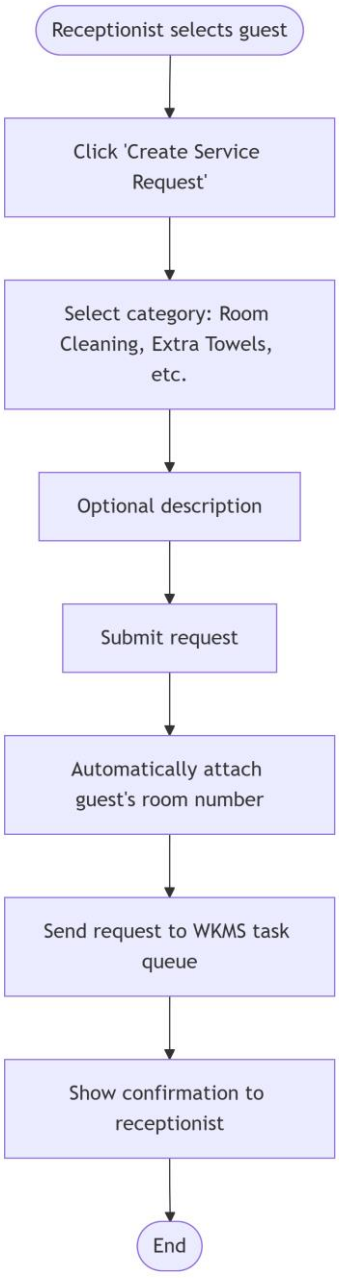
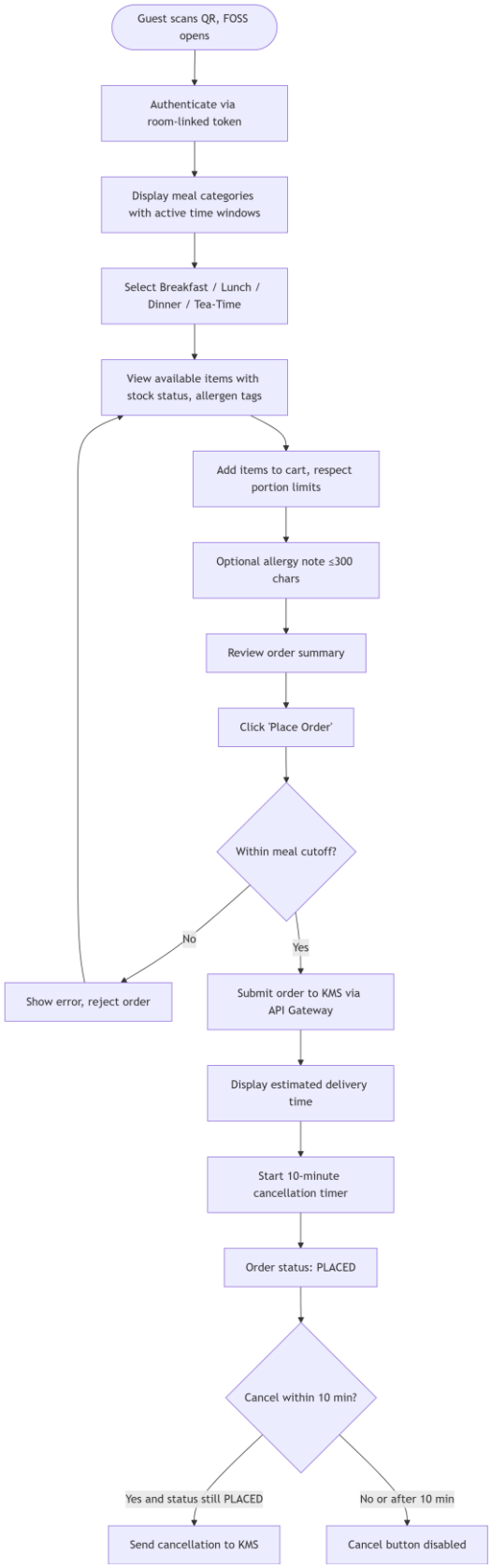


Figure 28 - FDS-15: Service Request Creation on Behalf of Guest Flow

5.2.4 Guest App / FOSS - User Flows

Only checked in guests authenticated via a room linked QR token are permitted to access the Guest App. Three flows are defined as covering food ordering, service request submission and tracking, and complaint filing.


Guest scans QR, FOSS opens
Authenticate via room-linked token
Display meal categories with active time windows
Select Breakfast / Lunch / Dinner / Tea-Time
View available items with stock status, allergen tags
Add items to cart, respect portion limits
Optional allergy note ≤300 chars
Review order summary
Click 'Place Order'
Within meal cutoff?
No
Yes
Show error, reject order
Submit order to KMS via API Gateway
Display estimated delivery time
Start 10-minute cancellation timer
Order status: PLACED
Cancel within 10 min?
Yes and status still PLACED
No or after 10 min
Send cancellation to KMS
Cancel button disabled

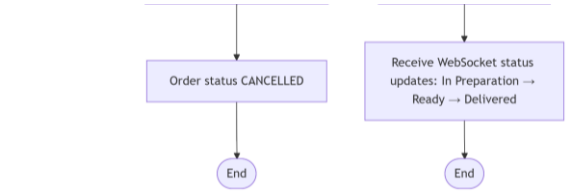


Figure 29 - FOSS-16: Food Ordering Flow

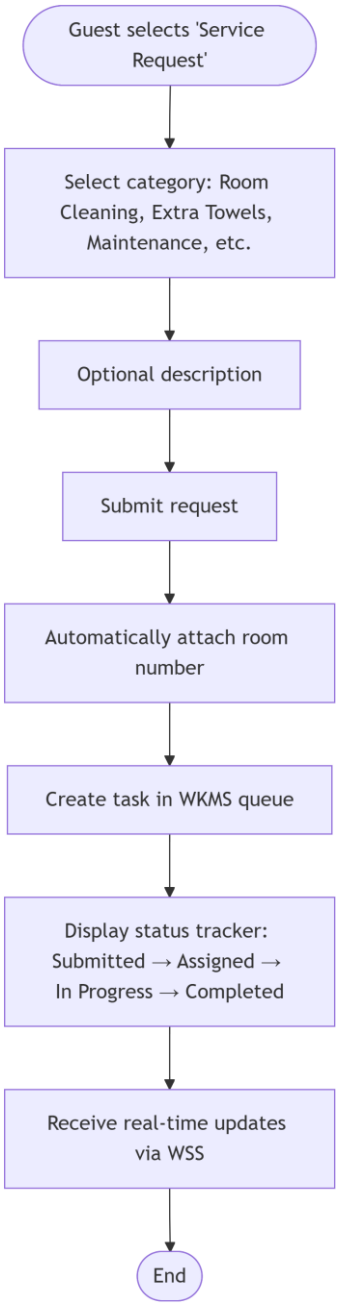


Figure 30 - FOSS-17: Service Request Submission and Tracking Flow

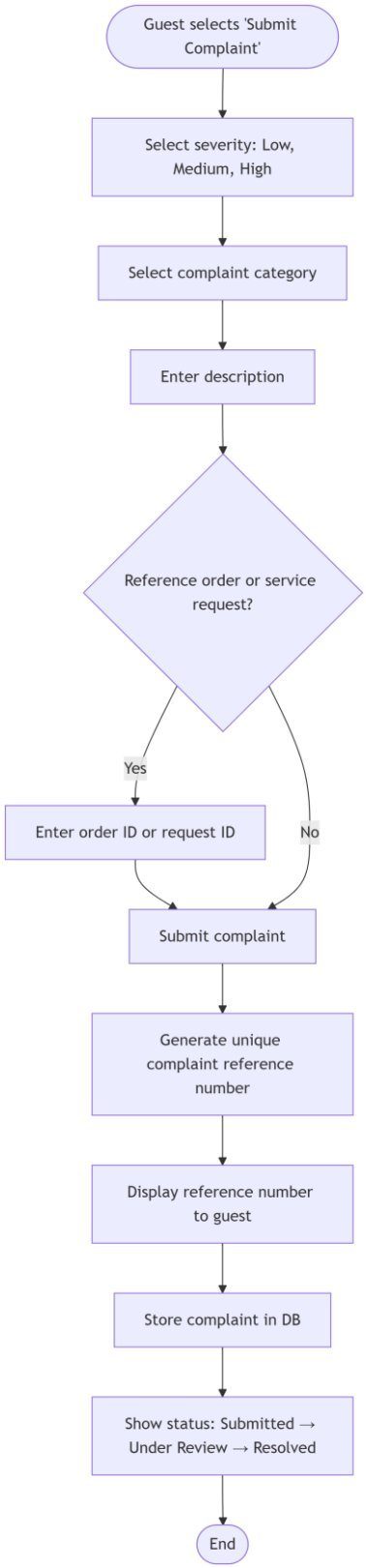
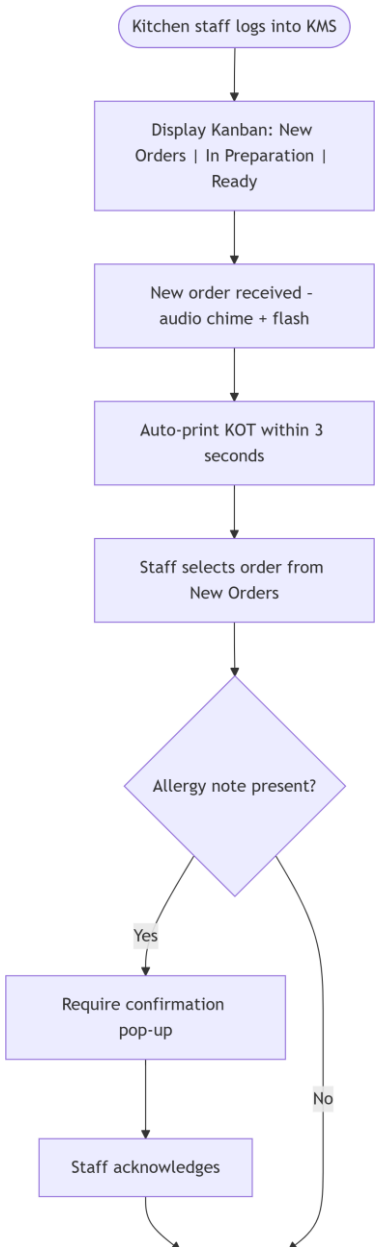


Figure 31 - FOSS-18: Complaint Filing and Tracking Flow

5.2.5 Kitchen Management System (KMS) - User Flows

The Kitchen Staff and Kitchen Manager roles are served by the Kitchen Management System. Three flows are defined covering food order processing, menu and stock management, and handling order cancellations received from FOSS.



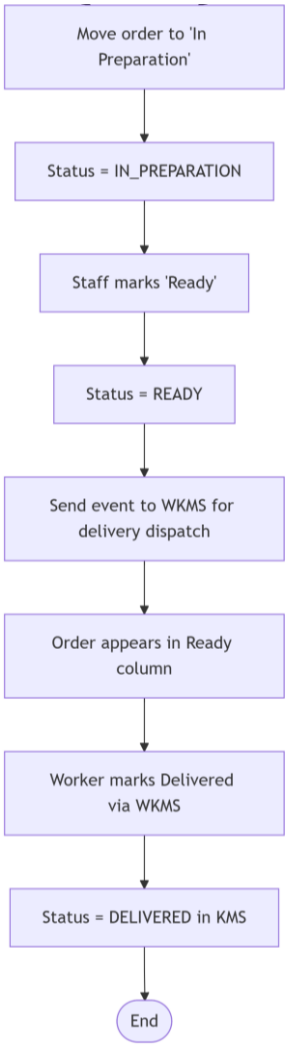


Figure 32 - KMS-19: Order Processing Flow

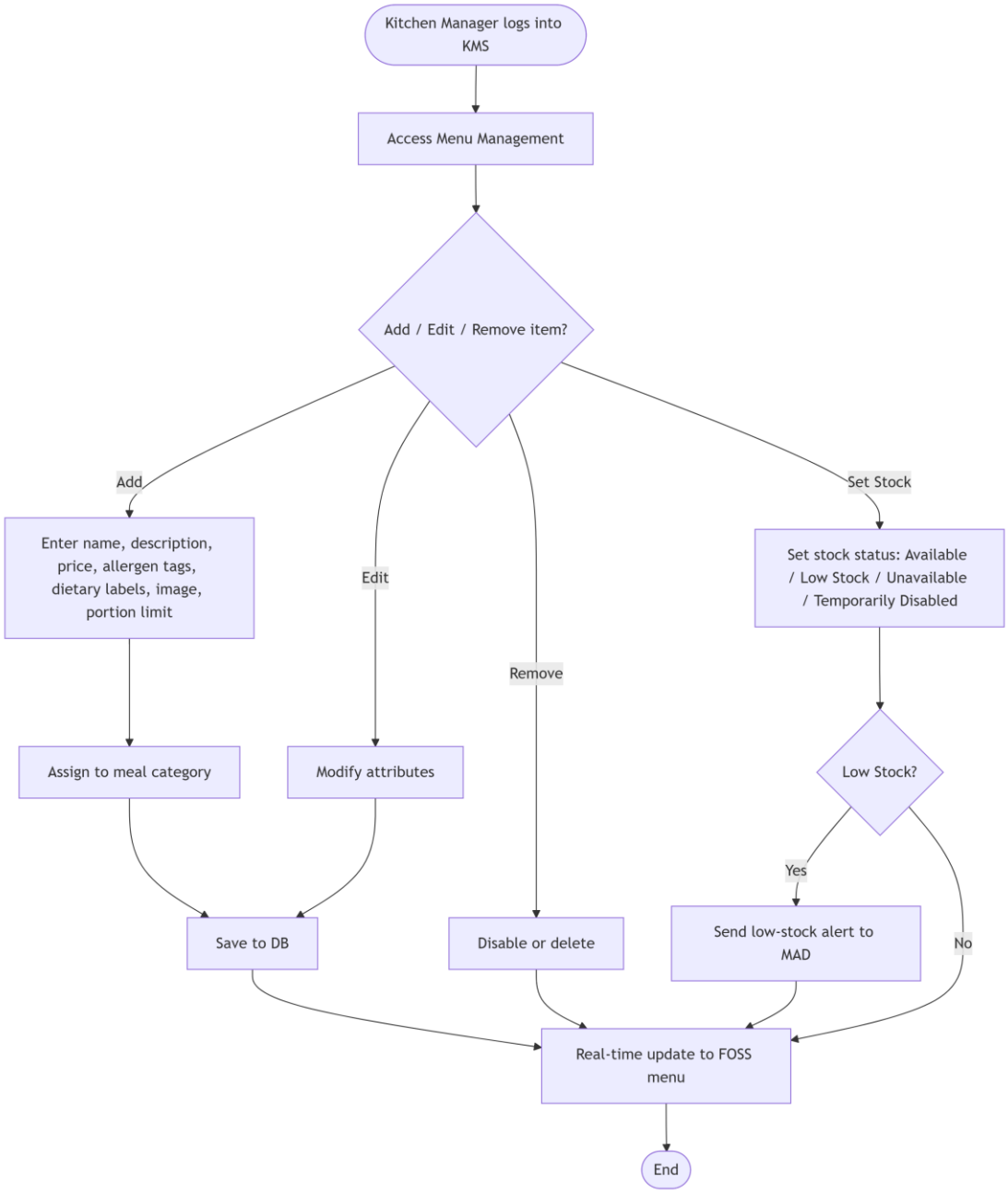


Figure 33 - KMS-20: Menu and Stock Management Flow (Kitchen Manager)

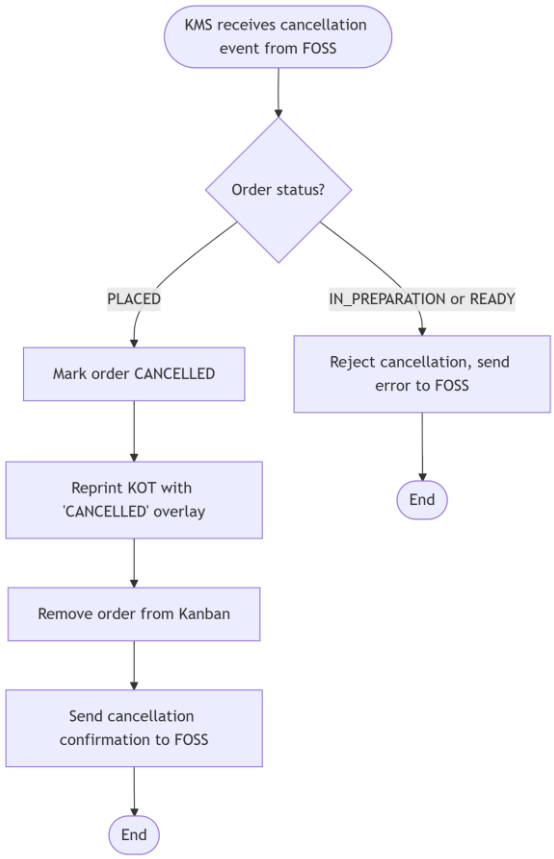


Figure 34 - KMS-21: Order Cancellation Received from Guest Flow

5.2.6 Worker Management System (WKMS) - User Flows

The Worker role is served by the Worker Management System, which operates as a Progressive Web Application on mobile devices. Three flows are defined covering general task claim and completion, the specialized food delivery task, and the shift performance dashboard.

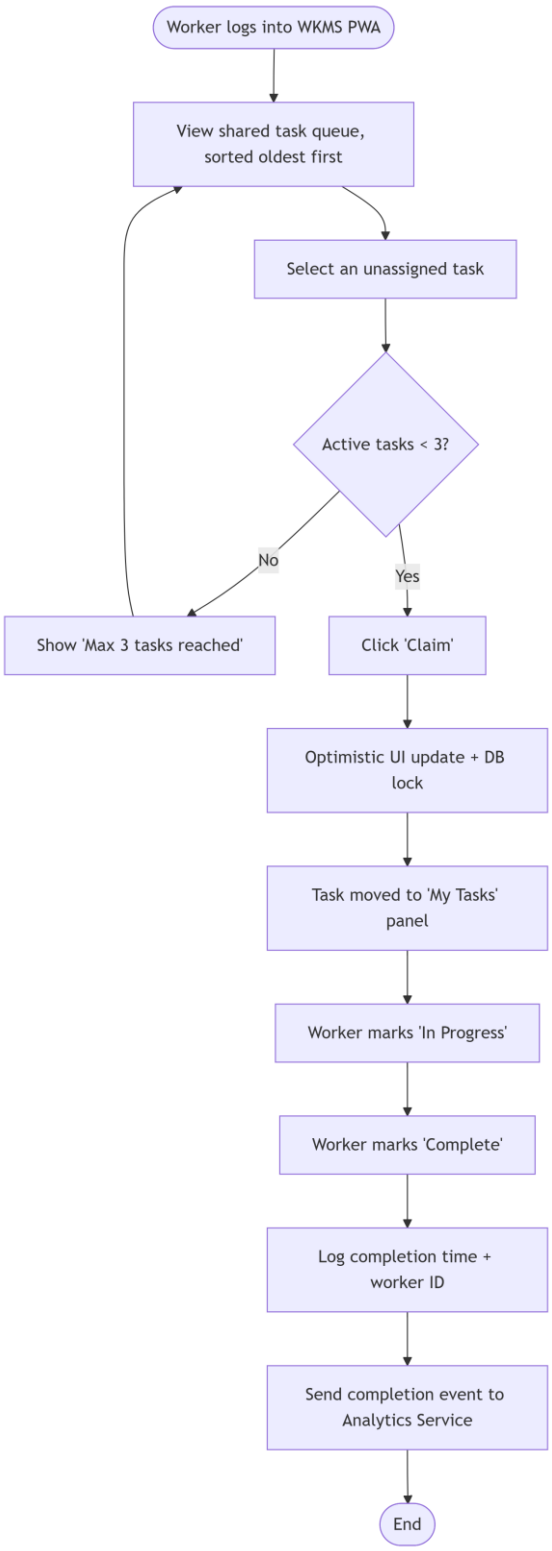


Figure 35 - WKMS-22: Task Claim and Service Completion Flow

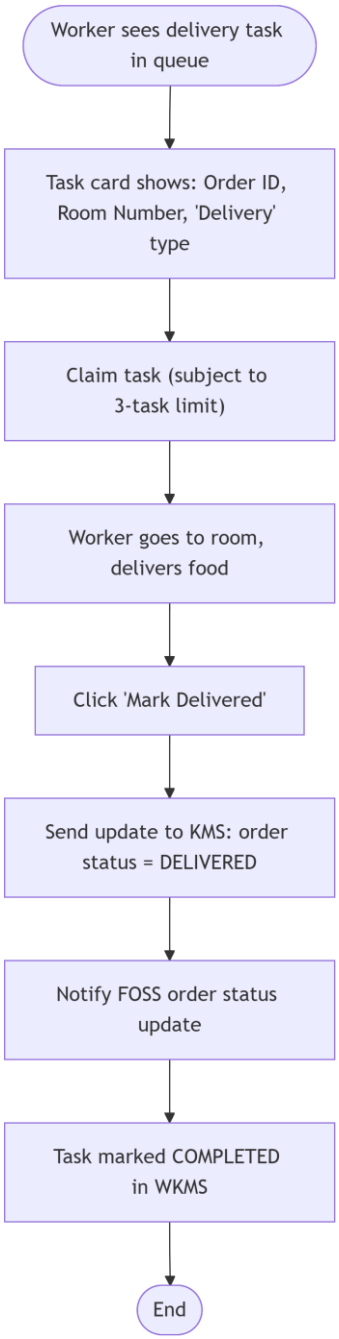


Figure 36 - WKMS-23: Food Delivery Task Flow

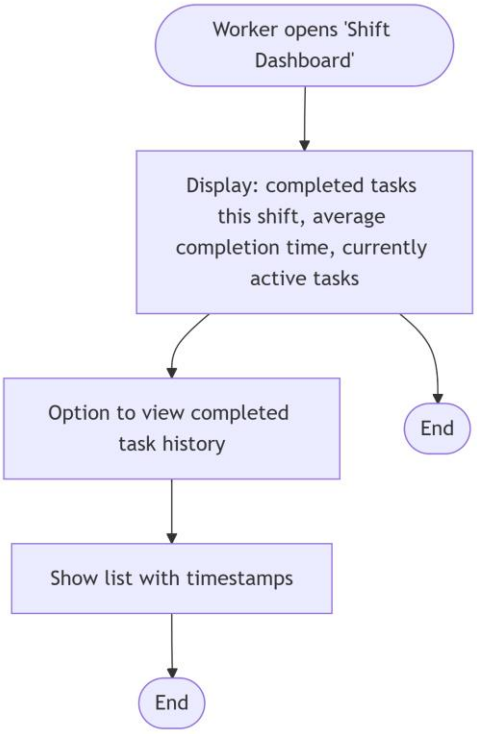


Figure 37 - WKMS-24: Shift Dashboard and Performance View Flow

5.3 Accessibility Considerations (WCAG 2.1 Compliance)

The Forever Hotel system is designed to comply with the Web Content Accessibility Guidelines (WCAG) 2.1 [15] at Level AA across all six subsystems. Accessibility requirements have been incorporated at the design stage to ensure that all users — including those with visual, motor, auditory, and cognitive disabilities — can perceive, operate, understand, and access the system's content and functionality. The requirements are structured according to the four WCAG 2.1 principles: Perceivable, Operable, Understandable, and Robust (POUR).

1. Perceivable

- All images need descriptive alt text.
- Form fields require visible <label> (not just placeholders).
- Colour alone is never used to convey information (e.g., room status includes text labels).
- Text contrast ≥4.5:1 (body) / 3:1 (large text).
- Pages reflow at 320px without horizontal scrolling.
- Non-text UI elements have 3:1 contrast.

2. Operable

- Full keyboard operability (no mouse required).
- Focus visible on all interactive elements.
- Touch targets ≥44×44px (buttons, claims, etc.).
- 10-minute order cancellation includes a live timer + extension warning.
- No flashing more than 3 times/second (KMS alert).
- Logical focus order matches visual sequence.
- On devices that support vibration, the Claim button shall trigger haptic feedback when a task is successfully claimed

3. Understandable

- HTML root declares lang='en'.
- Consistent navigation across each subsystem.
- Error messages are specific and appear next to the field + announced via ARIA.
- Booking flow includes a review page before final submission.

4. Robust

- Valid HTML (no duplicate IDs, proper nesting).
- Interactive components expose name, role, state via ARIA.
- Live regions (aria-live="polite" / role="alert") announce real-time updates (task queues, escalations).

CHAPTER 6: SECURITY DESIGN

The security architecture of the Forever Hotel system is designed to protect guest data, prevent unauthorized access, and ensure compliance with applicable security standards. All security mechanisms are implemented consistently across the six microservices and are enforced centrally where possible.

6.1 Authentication Mechanism

Authentication in the Forever Hotel system is implemented using JSON Web Tokens (JWT) issued by a centralized Authentication Service. Three distinct authentication flows are supported, corresponding to the different user categories accessing the system.

6.1.1 JWT-Based Authentication

The Authentication Service is responsible for verifying user credentials and issuing signed JWT tokens. All subsequent requests to protected endpoints must include the JWT token in the HTTP Authorization header using the Bearer scheme.

Table 10 - JWT Token Specifications

Parameter	Specification
Signing algorithm	HS256 (HMAC-SHA256)
Token expiry (staff)	8 hours
Token expiry (guest)	24 hours or until checkout
Token refresh mechanism	New login required after expiry
Storage (client-side)	HTTP-only cookie or localStorage
Inactivity timeout	token invalidated after 24h of no requests. new login required.

The JWT token contains the following claims

- Issuer(iss) - Authentication Service identifier
- Subject(sub) - User ID (UUID)
- Expiration Time(exp) -Token expiry timestamp

- Issued At(iat) - Token issuance timestamp
- Custom claim(role) - User role (GUEST, RECEPTIONIST, KITCHEN_STAFF, KITCHEN_MANAGER, WORKER, MANAGER)
- Room Number (Custom claim, guests only) - Room number associated with the guest session

6.1.2 Guest Authentication Flow

Guest users access the Food Ordering and Service Request System (FOSS) by scanning a QR code displayed in their hotel room.

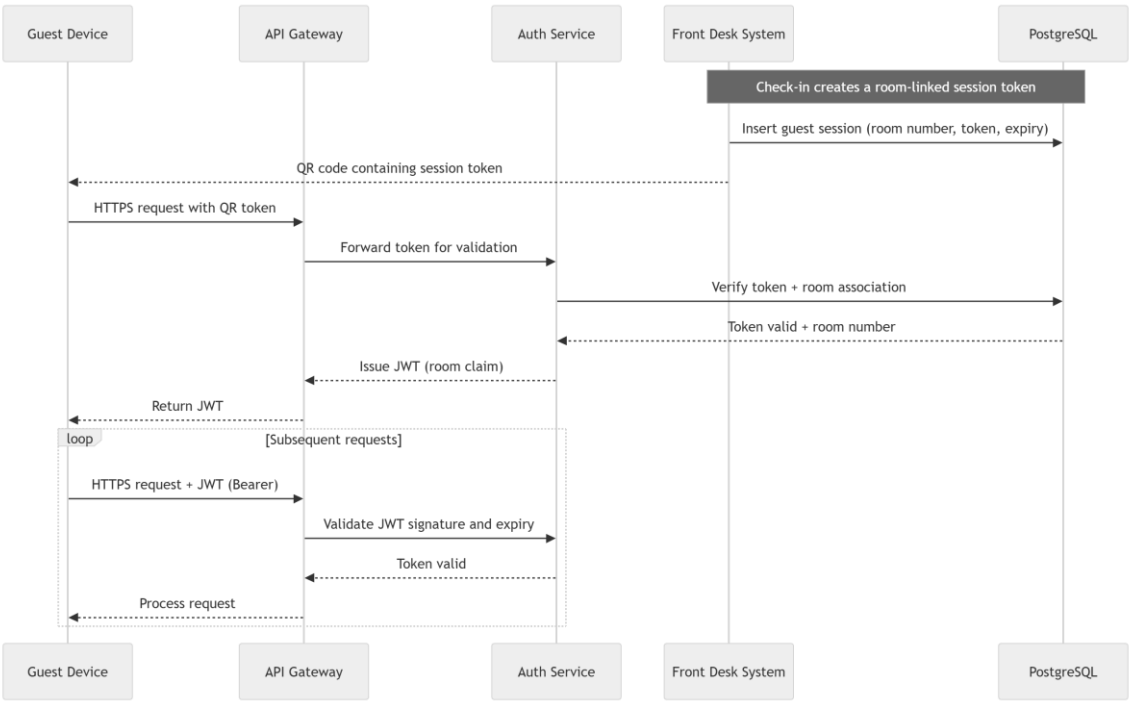


Figure 38 - Guest Authentication Flow

6.1.3 Staff Authentication Flow

Staff users (Manager, Receptionist, Kitchen Staff, Worker) receive auto-generated credentials via email when their accounts are created by the Manager. Upon first login, a password change is enforced.

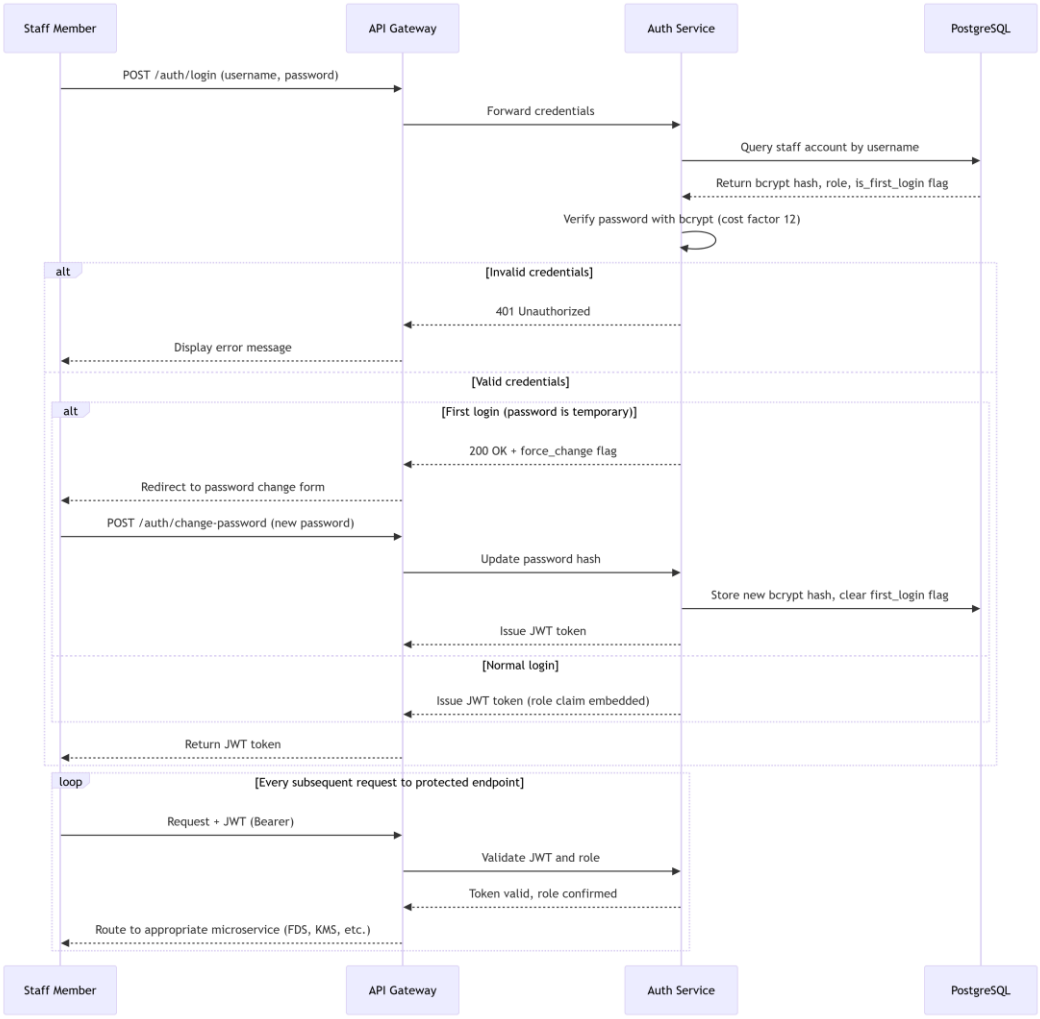


Figure 39 - Staff Authentication Flow

6.1.4 Password Storage

All user passwords are stored as bcrypt [16] hashes with a minimum cost factor of 12, as required by NFR-11 of the SRS [28]. Plain-text passwords are never stored in the database or written to logs. The bcrypt algorithm incorporates a salt automatically, protecting against rainbow table attacks.

6.2 Authorization Levels

Role-Based Access Control (RBAC) is enforced throughout the Forever Hotel system. Each user is assigned a single role at account creation, and all protected endpoints validate that the authenticated user possesses the required role before processing the request.

6.2.1 Role Definitions

Table 11 - System Roles and Permissions

Role	**Accessible Subsystems**	**Key Permissions**
GUEST	FOSS	Place food orders; submit service requests; submit complaints; track request status
RECEPTIONIST	FDS	Check-in/out; walk-in bookings; room status management; escalate tasks; view audit log
KITCHEN_STAFF	KMS	View order queue; update order status; acknowledge allergy information
KITCHEN_MANAGER	KMS, MAD (partial)	Menu management; stock status; ordering time windows; low-stock alerts
WORKER	WKMS	Claim tasks; update task status; view assigned tasks; complete deliveries

MANAGER	MAD, FDS (read-only), WKMS (read-only)	Analytics; promotions; staff account management; complaint resolution; report export

6.2.2 Permission Validation Process

For every authenticated request to a protected endpoint, the following validation sequence is executed:

1. The API Gateway extracts the JWT token from the Authorisation header
2. The token signature is verified using the shared HS256 secret
3. The exp claim is checked to ensure the token has not expired
4. The role claim is extracted and compared against the endpoint's required role
5. If all checks pass, the request is routed to the target microservice with the user context forwarded in request headers

For guest-specific endpoints (FOSS), an additional check is performed to ensure the roomNumber claim corresponds to an active, checked-in stay. This validation prevents cross-room access to guest data.

6.3 Data Protection Approach

The Forever Hotel system implements a multi-layered data protection strategy covering payment data, personally identifiable information (PII), and operational data.

6.3.1 Payment Data Protection (PCI-DSS Compliance)

All payment processing is handled exclusively through Stripe [17], as specified in the SRS (SI-01). The Forever Hotel system never receives, processes, or stores raw payment card data. This architecture ensures PCI-DSS SAQ A compliance.

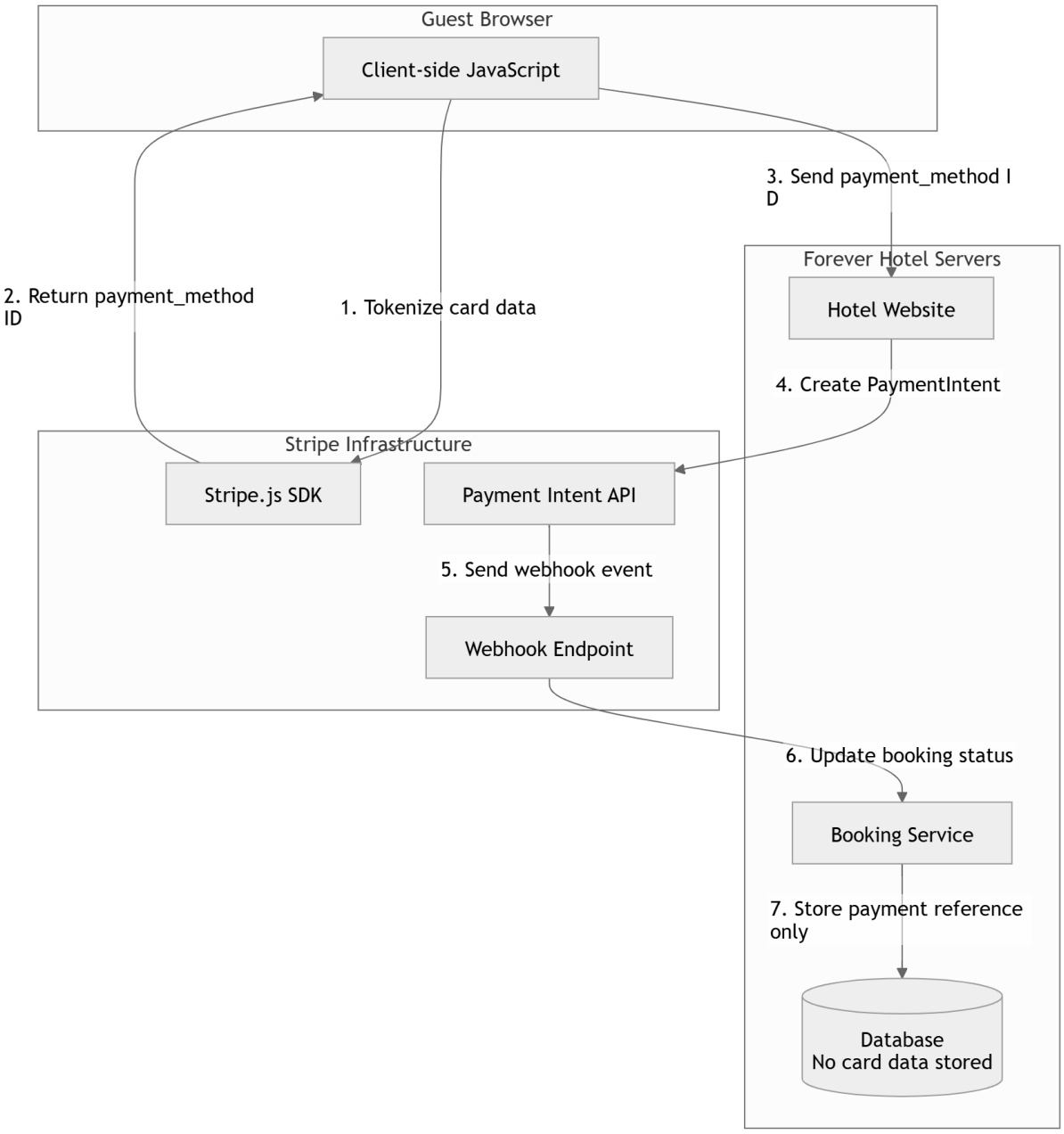


Figure 40 - Payment Data Protection

Key protections enforced:

- Raw card data is never transmitted to Forever Hotel servers
- Only Stripe-issued payment references are stored in the database
- Stripe webhooks are validated using signature verification
- All payment-related API calls use TLS 1.2+ encryption

6.3.2 Personally Identifiable Information (PII) Protection

Guest PII is protected through encryption, access restriction, and data minimization principles.

Table 12 - PII Protection Measures

Data Element	Protection Mechanism	Access Restriction
Guest name	Database encryption at rest	Authenticated staff only
Email address	Database encryption at rest	Guest (own), Manager
Phone number	Database encryption at rest	Guest (own), Receptionist
NIC/Passport scan	Encrypted object storage	Receptionist only
Room number	Not stored with PII in logs	Role-based API access

PII handling rules enforced across all services:

- PII is excluded from RabbitMQ event payloads (opaque UUID references used instead)
- PII is excluded from system logs (redaction applied before log emission)
- Guest sessions are isolated by room number (no cross-room data access)
- Right-to-erasure workflow: upon guest request, personal data is anonymised while financial records are retained for compliance

6.3.3 Audit Logging

A tamper-evident audit log is maintained for all security-relevant actions, as required by FD-16 of the SRS.

Table 13 - Audit Log Events

Event Category	Logged Actions	Retention Period
Authentication	Login attempts (success/failure), password changes, token issuance	12 months

Front desk operations	Check-in, check-out, room changes, task assignments	3 years
Payment events	Payment confirmation, refunds, failures	7 years
Staff account management	Account creation, role changes, deactivation	3 years
Data access	PII view/export operations	12 months
Task/Service Records	Service request, Complaints	1 year
Food Order Records	Order records, Cancelation records	2 year

Audit logs are stored in a write-once, append-only table in PostgreSQL. Deletion of audit log entries is not permitted by the application.

6.4 Input Validation and Sanitization Strategy

All input received from client applications is validated and sanitised before processing. The strategy follows a defense-in-depth approach with validation performed at multiple layers.

6.4.1 Validation Layers

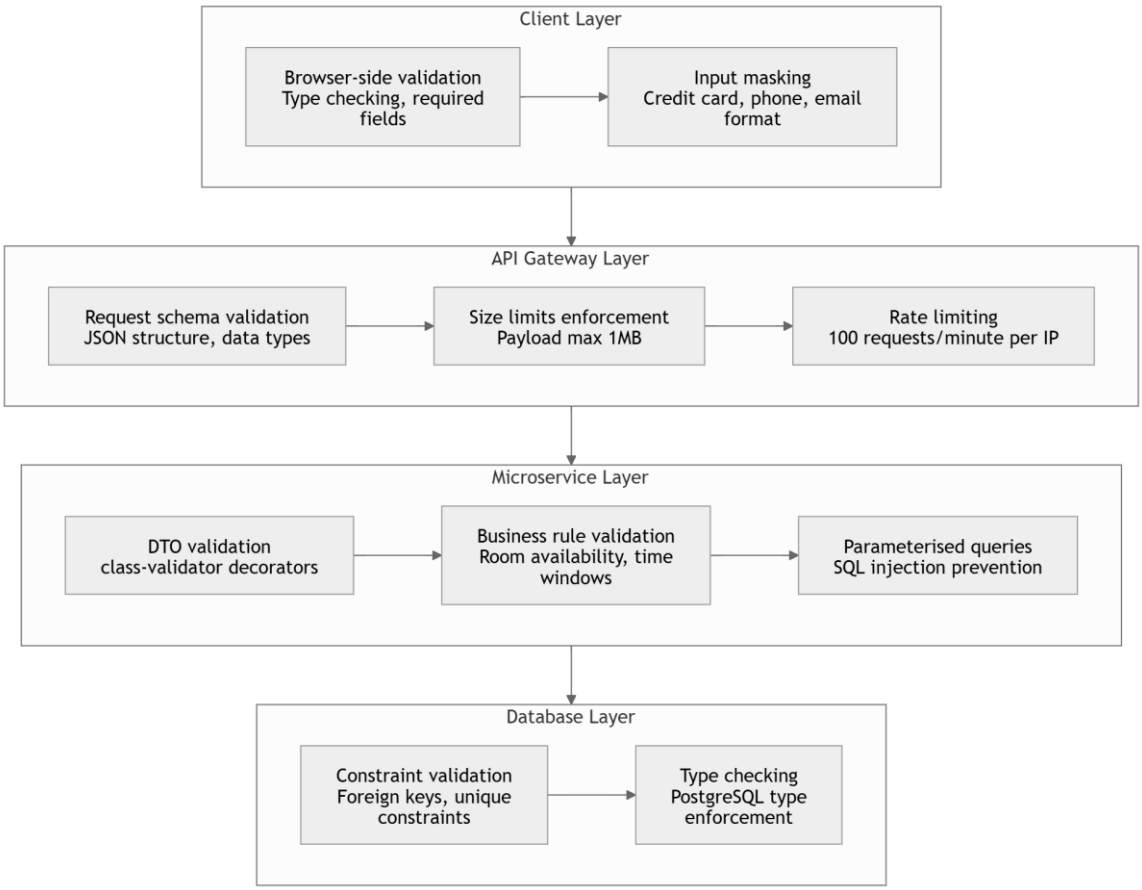


Figure 41 - Multi-Layer Input Validation Architecture

6.4.2 Validation Rules by Data Type

Table 14 - Input Validation Specifications

Data Type	Validation Rules	Sanitisation
Email (RFC 5321)	Must contain @ and domain; max 320 characters	Trim whitespace; convert to lowercase
Phone number	Numeric only; 10-15 digits; optional + prefix	Strip non-numeric characters except +
Guest name	Letters, spaces, hyphens, apostrophes only; max 255 characters	HTML entity escape

Description text (service request, complaint)	Max 300 characters (FOSS-05, FOSS-10); no executable content	HTML entity escape; strip script tags
Room number	Alphanumeric; max 10 characters	Uppercase conversion
Monetary amounts	Integer in smallest currency unit (LKR cents); positive only	Parse to integer; reject decimals
Date/time (ISO 8601)	Valid UTC timestamp; check-in before check-out	Parse to UTC; reject invalid dates
Promotional code	Alphanumeric, hyphens, underscores; max 50 characters	Uppercase conversion; trim whitespace

6.4.3 SQL Injection Prevention

All database queries use parameterized queries (prepared statements) exclusively. String concatenation for SQL query construction is prohibited in the codebase. The NestJS TypeORM integration enforces parameterized queries by default.

6.4.4 Cross-Site Scripting (XSS) Prevention

All user-generated content displayed in web interfaces is HTML-entity escaped before rendering. Next.js [18] provides automatic escaping of values embedded in JSX templates. For rich text fields, a Content Security Policy (CSP) header is configured to restrict script execution to trusted sources only.

6.5 Encryption Standards for Data at Rest and in Transit

The Forever Hotel system employs industry-standard encryption protocols for all data, whether stored in databases or transmitted across networks.

6.5.1 Encryption in Transit

All external communications are encrypted using TLS 1.2 or higher, as required by NFR-10 of the SRS. TLS 1.0 and 1.1 are explicitly disabled.

Table 15 - Transport Encryption Specifications

Communication Path	Protocol	Port	Certificate
Client → API Gateway	HTTPS (TLS 1.2/1.3)	443	Let's Encrypt (auto-renewed)
Guest browser → Stripe	HTTPS (TLS 1.2+)	443	Stripe-managed
API Gateway → Microservices	HTTP (internal Docker network)	Varied	Not required (isolated network)
Microservice → PostgreSQL	TLS-enabled PostgreSQL wire protocol	5432	Self-signed (internal)
Microservice → RabbitMQ	TLS-enabled AMQP	5671	Self-signed (internal)
Printing Service → Printer	Raw TCP (port 9100)	9100	Not applicable (LAN only)

Internal network justification: Communication between containers within the Docker bridge network uses plain HTTP because the network is isolated from external access. No public routes exist to internal service ports. This approach reduces certificate management overhead while maintaining security through network isolation.

6.5.2 TLS Certificate Management

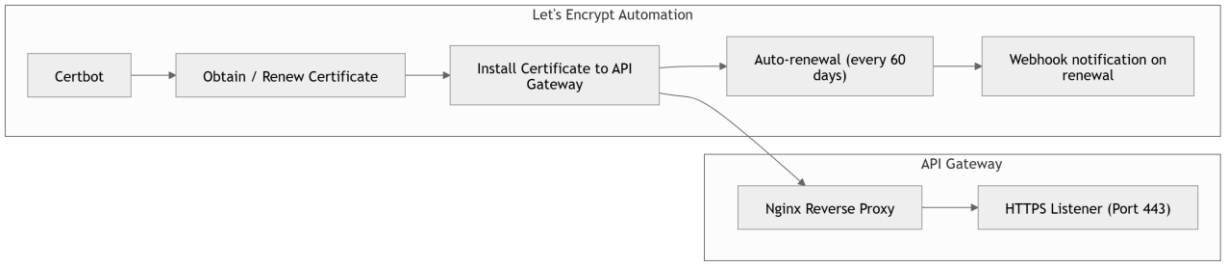


Figure 42-TLS Certificate Auto-Renewal Process

6.5.3 Encryption at Rest

Sensitive data stored in databases and object storage is encrypted using AES-256, the industry standard for data-at-rest encryption.

Table 16 - Data at Rest Encryption Specifications

Storage Location	Data Encrypted	Encryption Method	Key Management
PostgreSQL database	Guest PII (name, email, phone, NIC)	AES-256 (column-level)	Environment variable (rotatable)
PostgreSQL database	Passwords (bcrypt hashes)	bcrypt (cost factor 12)	N/A (hash only)
Object storage (ID scans)	NIC/passport scan images	AES-256 (file-level)	Cloud provider managed
Container volumes	Log files	Filesystem-level encryption (LUKS)	Server-managed
Backup archives	Full database backup	AES-256 (backup tool)	Separate key storage

6.5.4 Database Column Encryption

Sensitive columns in the PostgreSQL database are encrypted using the pgcrypto extension with AES-256 in GCM mode. The encryption key is stored in an environment variable on the database server and is never committed to source control.

Access to decrypted data is restricted to authorized services through the API layer. Raw database access does not yield readable PII.

CHAPTER 7: DEPLOYMENT DESIGN

The deployment design of the Forever Hotel system defines the hosting environment, hardware specifications, network topology, environment separation, and continuous integration and delivery pipeline. All decisions are aligned with the cloud-native, containerized architecture described in Chapter 1 and the non-functional requirements documented in the SRS.

7.1 Hosting Plan

The Forever Hotel system is hosted on a Linux-based Virtual Private Server (VPS) provided by a commercial cloud infrastructure provider (e.g., DigitalOcean, Linode, or AWS Lightsail). All services are packaged as Docker containers and orchestrated using Docker Compose. This approach ensures environment parity across development, staging, and production, as required by NFR-19.

Table 17 - Hosting Plan Summary

Component	Hosting Approach	Justification
API Gateway	Docker container on VPS	Centralized request routing and TLS termination
Six subsystem microservices	Docker containers on VPS	Independent scaling and deployment
PostgreSQL database	Docker container with volume mount	Persistent data storage with backup capability
RabbitMQ message broker	Docker container with volume mount	Durable queues and event persistence
WebSocket server	Docker container on VPS	Real-time bidirectional communication

Component	Hosting Approach	Justification
Printing Service	Docker container on VPS	Internal communication with kitchen printer
SSL/TLS certificates	Let's Encrypt (auto-renewal via Certbot) [19]	Zero-cost, trusted certificates

Production VPS configuration:

- Provider: DigitalOcean (or equivalent)
- Region: Singapore (lowest latency for Sri Lanka)
- Operating system: Ubuntu 22.04 LTS
- Docker Engine: 24.x or later
- Docker Compose: 2.x

7.2 Hardware Requirements

The following hardware specifications apply to the production VPS hosting the Forever Hotel system. On-premise hardware is limited to the kitchen thermal printer and optional front desk receipt printer.

7.2.1 Server Hardware (VPS)

Table 18 - Production Server Minimum Specifications

Component	Minimum Requirement	Recommended
CPU	4 x86-64 cores	8 cores
RAM	8 GB	16 GB
Storage	50 GB SSD	100 GB SSD

Component	Minimum Requirement	Recommended
Network bandwidth	1 Gbps	1 Gbps
Public IPv4 address	1	1 (static)

Rationale: The 4-core / 8 GB configuration supports up to 300 concurrent users (NFR-14) and processes at least 100 concurrent food orders (NFR-15). Storage accommodates container images, database volumes, and 12 months of structured logs (NFR-17).

7.2.2 On-premises Hardware

Table 19 - On Premise Hardware Requirements

Device	Quantity	Interface	Purpose
Kitchen thermal printer (ESC/POS compatible)	1	Ethernet (TCP port 9100)	Automatic KOT printing for every food order
Front desk receipt printer	1	USB or Ethernet	Printing registration cards and payment receipts
Kitchen display monitor (1080p)	1	HDMI	KMS Kanban board visibility
Staff workstations (MAD, FDS, KMS, WKMS)	As needed	Wi-Fi / Ethernet	Browser-based access to staff subsystems
Guest mobile devices	N/A	Wi-Fi	Access to FOSS and WKMS via PWA

Printer connectivity: The kitchen thermal printer must be assigned a static IP address on the hotel's local area network to allow the Printing Service to establish a raw TCP connection.

7.3 Network Diagram

The network architecture separates public-facing traffic from internal service communication. All external requests are terminated at the API Gateway, which then proxies requests to internal microservices over a Docker bridge network.

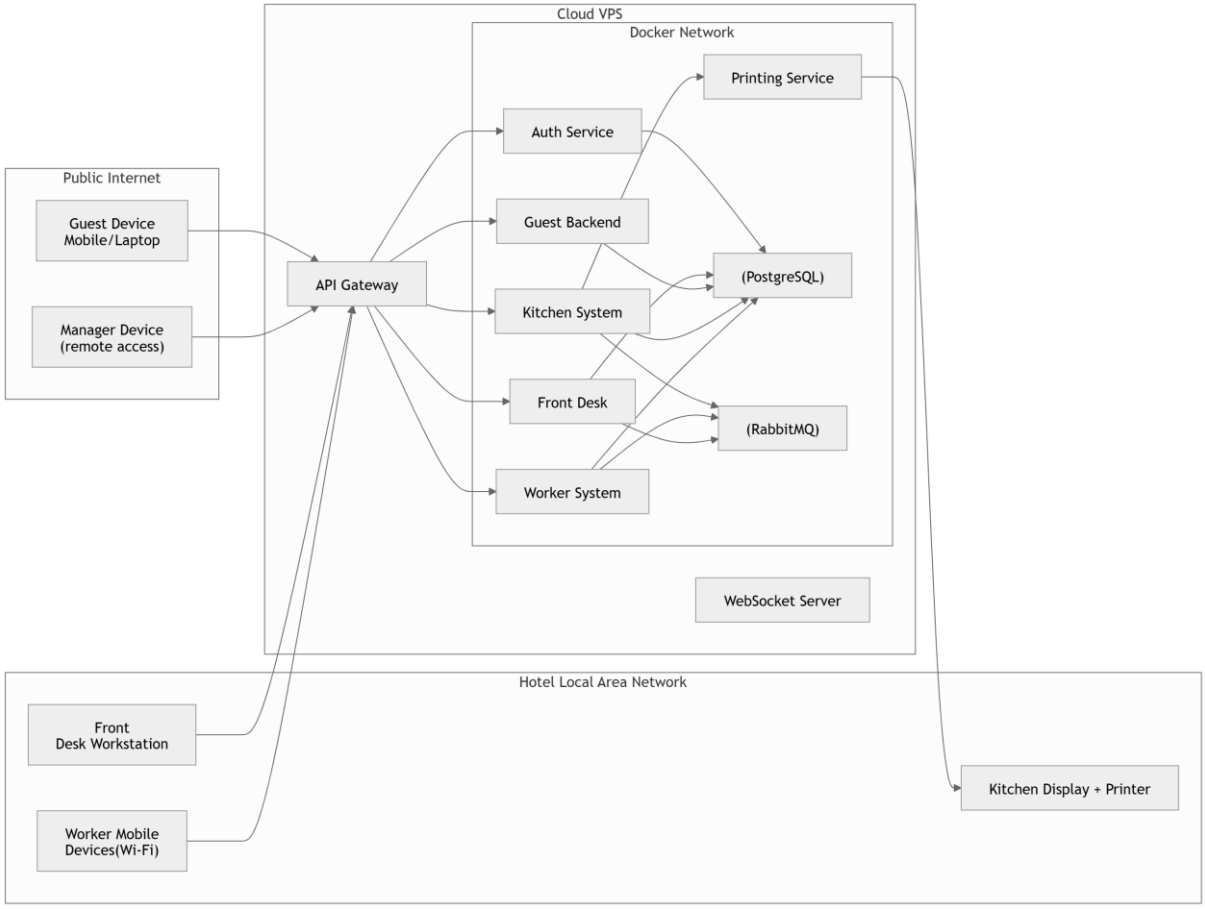


Figure 43 - Forever Hotel Network Diagram (Production Deployment)

Key network security controls:

- Only the API Gateway exposes port 443 to the public internet.
- All internal services communicate over a Docker bridge network with no public routes.

- Staff-facing subsystems (MAD, FDS, KMS, WKMS) are accessible only from the hotel's internal Wi-Fi (network-level restriction recommended, but not enforced by the application).
- The kitchen printer is reachable only from the Printing Service container via static LAN IP.

7.4 Environment Specification

Three distinct environments are maintained throughout the software development lifecycle: Development, Staging, and Production. Each environment runs the same Docker container images, ensuring environment parity.

Table 20 - Environment Specifications

Aspect	Development	Staging	Production
Purpose	Local development and unit testing	Integration testing, UAT, pre-production validation	Live hotel operations
Host	Developer workstation (Docker Desktop)	Separate VPS (same spec as production)	Production VPS (4 CPU / 8 GB RAM)
Database	PostgreSQL container (ephemeral, seeded with test data)	PostgreSQL container (persistent volume, anonymised production snapshot)	PostgreSQL container (persistent volume, production data)
Message broker	RabbitMQ container (default config)	RabbitMQ container (durable queues enabled)	RabbitMQ container (durable + dead-letter queues)

Aspect	Development	Staging	Production
External APIs	Stripe test mode, SendGrid sandbox, Booking.lk mock	Stripe test mode, SendGrid sandbox, Booking.lk sandbox	Stripe live mode, SendGrid live, Booking.lk production
Thermal printer	Simulated (print to file/log)	Simulated or test printer	Physical kitchen printer (static IP)
SSL/TLS	Self-signed certificate (localhost)	Let's Encrypt staging certificate	Let's Encrypt production certificate
Logging	Console (debug level)	JSON logs to file (info level)	JSON logs to file + remote syslog (warn/error level)
Access	Development team only	Hotel management and QA team	All hotel staff and guests

Environment promotion workflow: A container image that passes all tests in the staging environment is promoted unchanged to production using the same image tag. No code changes are permitted between staging and production.

7.5 CI/CD Pipeline Overview

A Continuous Integration and Continuous Delivery (CI/CD) pipeline [20] is implemented using GitHub Actions. The pipeline automates building, testing, and deployment of all six microservices. The goal is to enable independent deployment of each subsystem while maintaining system-wide reliability.

7.5.1 Pipeline Stages

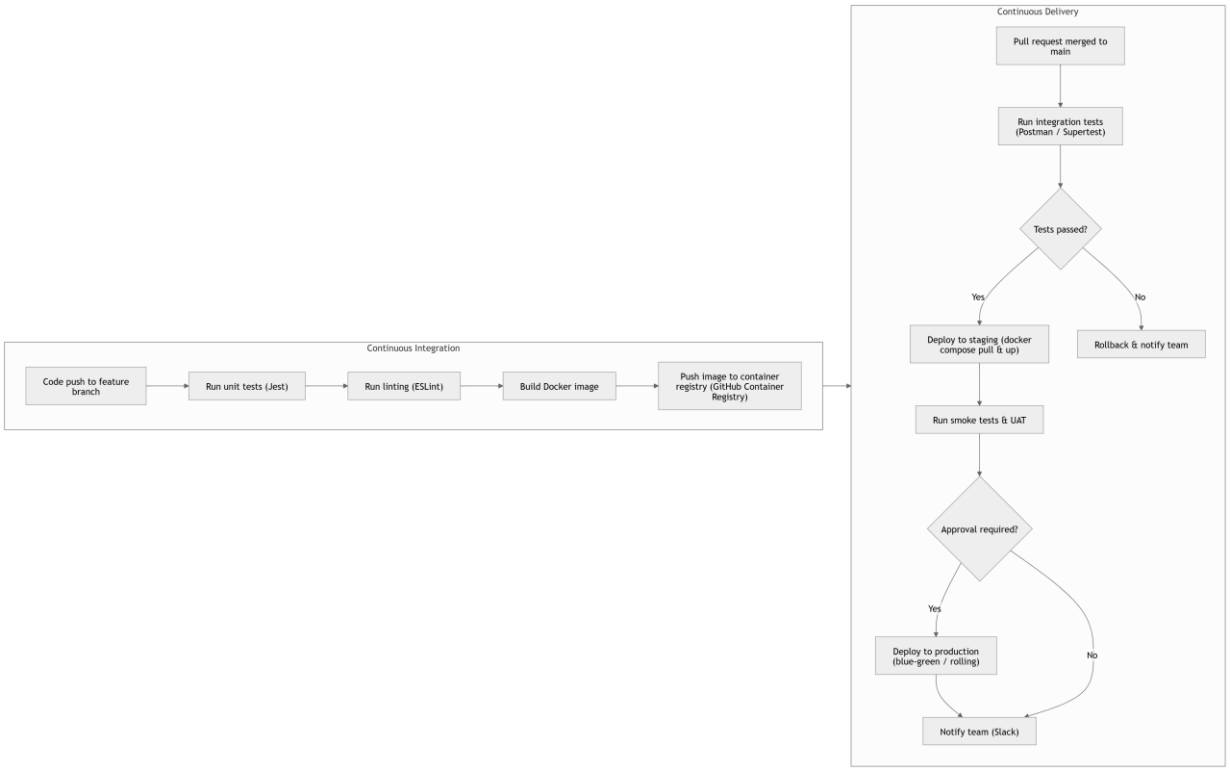


Figure 44 - CI/CD Pipeline Overview (GitHub Actions)

7.5.2 Pipeline Implementation Details

Table 21 - CI/CD Pipeline Specifications

Stage	Trigger	Actions	Success Criteria
Unit test	Push to any branch	Run service-specific unit tests with Jest	100% pass rate; coverage ≥ 80%
Lint	Push to any branch	Run ESLint on TypeScript code	Zero errors; zero warnings (strict mode)

Stage	Trigger	Actions	Success Criteria
Build image	Push to main or release branch	Build Docker image using Dockerfile; tag with commit SHA and latest	Image builds successfully; size < 500 MB
Push to registry	After successful build	Push to GitHub Container Registry (ghcr.io)	Image layers pushed without authentication errors
Integration tests	After merge to main	Run API contract tests, database migration tests, RabbitMQ connectivity tests	All tests pass; no regression
Deploy to staging	After integration tests pass	SSH into staging VPS; docker compose pull and up -d	Containers reachable; health checks pass
Smoke tests	After staging deployment	Verify critical endpoints (login, booking, order placement)	Response time < 500 ms; expected HTTP status
Deploy to production	Manual approval (manager or lead)	Blue-green deployment: new containers start, old drained, switch reverse proxy	Zero downtime; existing sessions preserved
Rollback	On deployment failure	Revert to previous image tag; restart containers	System returns to last known good state within 2 minutes

7.5.3 Independent Service Deployment

Because each microservice is containerized and communicates via well-defined APIs, the pipeline supports independent deployment of any subsystem without affecting others.

Independent deployment workflow:

1. Developer changes code only in the kms service folder.
2. CI pipeline builds only the kms Docker image.
3. CD pipeline deploys only the kms service to staging, then to production after approval.
4. RabbitMQ events and REST APIs [21] remain backward compatible; no coordination with other services is required.

This approach satisfies NFR16 (maintainability) and the independent deploy ability requirement stated in the architectural justification (Section 1.3)

CHAPTER 8: REFERENCES

[1] S. Newman, Building Microservices: Designing Fine-Grained Systems, Sebastopol, CA, USA: O'Reilly Media, 2021.

[2] P. G. D. Group, "PostgreSQL Official Website," 2023. [Online]. Available: https://www.postgresql.org/docs/.

[3] I. E. T. F. (IETF), "JSON Web Token (JWT)," 2015.

[4] D. Inc., "Docker Docs," 2023. [Online]. Available: https://docs.docker.com/.

[5] NGINX, "NGINX," 2022. [Online]. Available: https://www.nginx.com/learn/api-gateway/.

[6] I. E. T. F. (IETF), "The Transport Layer Security (TLS) Protocol Version 1.2," 2008.

[7] I. E. T. F. (IETF), "JSON Web Token (JWT)," 2015.

[8] VMware, "RabbitMQ Official Docs," 2023. [Online]. Available: https://www.rabbitmq.com/documentation.html.

[9] P. G. D. Group, "PostgreSQL Official Website," 2023. [Online]. Available: https://www.postgresql.org/docs/.

[10] P. Project, "PgBouncer," 2023. [Online]. Available: https://www.pgbouncer.org/.

[11] OASIS, "Advanced Message Queuing Protocol (AMQP) Version 1.0," 2012.

[12] OASIS, "Advanced Message Queuing Protocol (AMQP) Version 1.0," 2012.

[13] NestJS, "NestJS Documentation," 2023. [Online]. Available: https://docs.nestjs.com/websockets/gateways.

[14] F. H. D. Team, "Figma," Figma, [Online]. Available: https://www.figma.com/design/oHP2rxAfJCZeAB2aodrH8I/Forever-city-Hotel-Management-system?node-id=0-1&t=qFnkTQWVJaWjZz5S-1

[15] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," 2018.

[16] O. Project, "OpenBSD," 2023. [Online]. Available: https://man.openbsd.org/bcrypt.

[17] Stripe, "Stripe Docs," 2023. [Online]. Available: https://stripe.com/docs/security.

[18] Vercel, "Next.js," 2023. [Online]. Available: https://nextjs.org/docs.

[19] I. S. R. Group, "Let's Encrypt," 2023. [Online]. Available: https://letsencrypt.org/docs/.

[20] GitHub, "GitHub Docs," GitHub , 2023. [Online]. Available: https://docs.github.com/en/actions.